

CloudStore Documentation

Description	This documentation provides the fundamental information and core concepts of the CloudStore, as well as the guidelines and procedures required to begin integrating a service.
Generated	July 29, 2026
Author(s)	OVHcloud
Copyright	Copyright © 2026 OVHcloud

Table of Contents

Home

- CloudStore
- Contact Us

Explanation

Foundations

- Features
- Hosting the CloudStore
- CloudStore architecture
- Roles Overview

Identity & access

- CloudStore Authentication
- Service Authorization

Platform internals

- Networking
- Data & Storage
- Data classification & handling
- Security & Compliance
- Observability

Packaging & delivery

- What is a CloudStore service package ?
- Package delivery to customer platforms

How-to Guides

- Setup service authorization

Reference

Concepts & definitions

- Architecture Layers
- Persona Definitions
- Service Definitions
- Tenancy

Capabilities & operations

- Features Reference
- Platform runtime & deployment
- Availability & Disaster Recovery
- Roadmap & Maturity

- Glossary

1 Home

1.1 CloudStore

The **CloudStore** is a high-level infrastructure framework to manage datacenters, from a single rack up to a full DC. It ships with core services to assist DC operators and IT admins, a catalog of prepackaged software (VCF, etc.), and a clear API to package your own services through [Terraform](#).

CloudStore is currently deployed on top of **OPCP** — a complete solution for deploying, automating, and maintaining on-premise cloud computing systems.

New to CloudStore?

Start with [CloudStore features](#) to understand what the platform does, then [Hosting the CloudStore](#) to see where it fits in the stack.

1.1.1 What CloudStore gives you

- **A marketplace of services** — cloud providers deploy and manage Cloud Native solutions for their customers from a catalog.
- **A single, GitOps-driven platform** — a K3S cluster with core services (Keycloak, UI, API) reconciled declaratively from Git.
- **Multi-tenant isolation** — accounts, layered authentication, and service-driven authorization out of the box.
- **A packaging API** — build and integrate your own services as versioned OCI packages using Terraform/OpenTofu.

1.1.2 Explore the documentation

This documentation follows the [Diátaxis](#) model. Pick the entry point that matches what you need:

Explanation

Understand *how* CloudStore works and *why* — architecture, identity, networking, storage, and packaging concepts.

How-to Guides

Step-by-step guides to build a package, create a service, and set up authorization.

Reference

Look up precise details — personas, services, the feature matrix, and the glossary.

Contact us

Reach support for your platform through your OVHcloud point of contact.

1.2 Contact Us

This page describes how to reach support for your CloudStore.

1.2.1 Getting support

Your OVHcloud point of contact

Requests concerning your platform — delivery, configuration, incidents, or change requests — go through the **OVHcloud point of contact identified for your deployment**, using the support channels agreed for your contract.

For training or assistance implementing a solution on top of the platform, contact your sales representative or request an analysis of your project by the OVHcloud Professional Services team.

1.2.2 Support Guidelines

Tip

Provide sufficient technical context with every request — the environment concerned, the component involved, and the exact error message. Complete technical information helps the support team respond more efficiently and accurately.

2 Explanation

2.1 Explanation

Conceptual documentation that explains **how the CloudStore works and why it is built the way it is**. Start here if you want to understand the platform before operating or extending it.

The pages are grouped from the big picture down to the platform internals, and finally to how services are packaged and delivered.

2.1.1 Foundations

CloudStore features

What the platform does, how its features relate to each other, and the direction it is heading.

Hosting the CloudStore

Where CloudStore runs (OPCP Core or PaaS SecNumCloud) and how it fits into the three-layer stack.

CloudStore architecture

The K3S cluster, core services, and the GitOps model that reconciles the platform.

Roles overview

The CloudStore team and the actors involved in operating and extending the platform.

2.1.2 Identity & access

CloudStore authentication

The Keycloak-based, layered identity federation that establishes *who* a user is.

Service authorization

How each service declares its own permission model and how the platform provisions it.

2.1.3 Platform internals

Networking

Host network model, ingress, DNS, TLS & PKI, the CNI, and connectivity back to the infrastructure.

Data & Storage

Where CloudStore keeps its data, how it is persisted, and how it is backed up and recovered.

Security & Compliance

How tenants, secrets, and data are protected, and where the platform stands on SecNumCloud.

Observability

Metrics, logs, dashboards, and alerting rules shipped alongside the platform.

2.1.4 Packaging & delivery

Service package

What a CloudStore service package is and how it is structured.

Package delivery to customer platforms

How a published service package reaches an air-gapped customer platform, including the metapackage mechanism.

2.2 Foundations

2.2.1 Features

Introduction

The CloudStore is an OVHcloud product that enables cloud providers to deploy and manage Cloud Native solutions for their customers through a marketplace-based platform. This document explains the reasoning behind CloudStore's feature set, how features relate to each other, and the direction of the platform.

Out of scope today

CloudStore does not perform **metering, billing, or consumption tracking** today (there is no Agora integration). Consumption tracking and per-account quotas are on the [roadmap](#).

Why two distinct roles?

CloudStore serves two fundamentally different audiences with different needs:

- **Cloud providers** (IT admins, service admins) are OVHcloud customers who operate the platform. They provision infrastructure, deploy services, and manage accounts for their own customers. Their concern is operational: lifecycle management, capacity planning, and service reliability.
- **Cloud users** (end users) are the customers of cloud providers. They consume the services deployed for them. Their concern is functional: accessing the right applications with the right permissions.

This separation exists because cloud providers need full control over the platform without exposing operational complexity to their end users. The CloudStore (Layer 2) handles the provider-facing management, while the Landing Zone (Layer 3) provides a simplified, scoped interface for end users.

The controller/app pattern

Each service in CloudStore follows a controller/app architecture:

- The **controller** (also called controlplane) is deployed first. It manages the lifecycle of apps and handles cross-tenant orchestration. A single controller can manage multiple apps across different accounts.
- **Apps** (also called dataplane) are the workloads deployed for a specific account. Each app is isolated to one account and cannot be shared between accounts.

This pattern exists for two reasons:

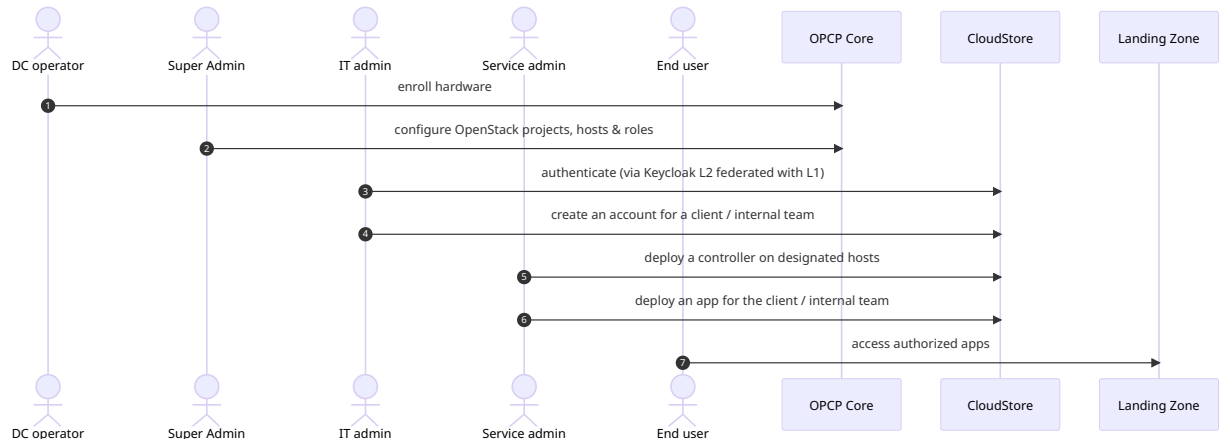
1. **Multitenancy:** A cloud provider can offer the same service to multiple customers, each getting their own isolated app instance while sharing the same controller infrastructure. This reduces operational overhead without compromising isolation.
2. **Lifecycle independence:** The controller and apps have separate lifecycles, so a controller can be reconfigured without affecting running apps, and apps can be scaled independently. (In-place service upgrade and rollback are planned — redeploy is the current path.)

How features build on each other

The CloudStore features follow a logical dependency chain that mirrors the platform's layered architecture:



Each step in this chain is performed by a specific persona:



- 1. Infrastructure enrollment:** Before anything can happen in CloudStore, hardware must be enrolled in OPCP Core by the DC operator.
- 2. OpenStack configuration:** The Super Admin configures OpenStack projects, assigns hosts and roles, and sets up IT admin access. This prepares the infrastructure for CloudStore to consume.
- 3. Platform access:** The IT admin authenticates to CloudStore via Keycloak (L2). Keycloak L2 is federated with OPCP Core's Keycloak (L1), which grants the necessary rights on OpenStack projects to manage infrastructure resources. This is the entry point for all management operations. The IT admin can then create service admins and delegate the management of specific services to them.
- 4. Account creation:** The IT admin creates accounts representing companies or departments. Each account gets its own Keycloak realm, providing complete IAM isolation. This must happen before deploying apps because apps are always scoped to an account.
- 5. Controller deployment:** The service admin activates a service, which deploys its controller on designated hosts. The controller uses Terraform resources orchestrated through Kubernetes (tofu-controller) to provision infrastructure on OpenStack. This is an asynchronous process.
- 6. App deployment:** Once a controller is active and an account exists, the service admin deploys apps for that account. Like the controller, the app uses Terraform resources orchestrated through Kubernetes to provision its workload (dataplane) on OpenStack. The deployment also creates Keycloak clients, roles, and groups in the account's realm, ensuring that only authorized users of that account can access the app.
- 7. End user access:** End users authenticate through their account's Keycloak realm (L3) and access their authorized apps via the Landing Zone. The Landing Zone retrieves the list of accessible apps by calling CloudStore APIs and filtering based on user permissions.

The security model

CloudStore's IAM design is built on Keycloak federation across two levels:

- **Keycloak L2 (CloudStore level):** Authenticates cloud providers (IT admins, service admins). The federation with OPCP Core's Keycloak grants the necessary rights on OpenStack projects.

- **Keycloak L3** (Landing Zone level): One realm per account, providing complete isolation between tenants. Each realm has its own users, roles, and groups.

When an app is deployed, service-specific roles and groups (e.g., `read`, `write`, `admin`) are created in the account's realm. Companies can create functional groups that reflect their organizational structure, with child groups mapping to these technical groups.

This approach was chosen because it provides strong tenant isolation by default — accounts cannot see or interact with each other's resources — while allowing flexible permission management within each account. Note that this isolation is enforced at the IAM realm and Kubernetes namespace levels; there is currently no per-tenant network, storage, or quota segregation (see [Tenancy](#)).

CloudStore platform upgrades

The CloudStore platform can be **upgraded in place**, and each upgrade automatically creates a backup of the CloudStore's **global state**.

There is intentionally **no rollback** — reverting an upgrade is neither planned nor desired. Recovery from a failed or unwanted upgrade is instead achieved by **reinstalling the cluster from scratch and restoring the global-state backup** taken during the upgrade.

Scope

This applies to the **CloudStore platform** itself. In-place upgrade (and rollback) of individual **services** is a separate capability that is still planned — see the controller/app lifecycle note above and the [roadmap](#) below.

Roadmap direction

The current feature set covers the essential lifecycle operations: deploying services, managing accounts, and providing access. The planned features extend this foundation in two directions:

For cloud providers — operational capabilities:

- **Backup / restore:** Protect service state and recover from failures
- **Upgrades:** Evolve services without disruption
- **Quotas:** Control resource consumption per account
- **Consumption tracking:** Monitor usage across accounts

For cloud users — self-service capabilities:

- **Unified observability:** Monitoring and metrics for deployed services
- **Secret management (KMS):** Centralized key and secret management
- **Consumption tracking:** Visibility into their own resource usage

2.2.2 Hosting the CloudStore

The CloudStore needs to be installed on a Kubernetes cluster. As far as hardware is concerned, this cluster can be hosted on On-Premise Cloud Platform (OPCP) or on Platform as a Service SecNumCloud (SNC CloudPlatform), with different prerequisites.

Where CloudStore fits

CloudStore is deployed **on top of OPCP Core** and sits in a three-layer model:

Layer	Name	Role
L1	OPCP Core	Hardware enrollment, OpenStack, datacenter operations — the foundation CloudStore consumes.
L2	CloudStore	Provider-facing management platform: the service marketplace, IAM, packaging and deployment.
L3	Landing Zone Manager	Simplified, scoped self-service interface for end users.



The L2/L3 split is logical, not physical

CloudStore (L2) and the Landing Zone Manager (L3) run on the **same single K3s node** — the layering separates personas and IAM scope, not hardware. Only L1 (OPCP Core) is a physically distinct platform.

The full **OPCP** product bundles the hardware (racks with embedded orchestration), **OPCP Core** (L1), and **CloudStore** (L2), delivered into a customer datacenter. **SNC CloudPlatform** is the SecNumCloud-qualified variant that OVHcloud hosts and operates itself.

On-Premise Cloud Platform (OPCP)

To be hosted on On-Premise Cloud Platform (OPCP), the CloudStore and thus the services it integrates should be able to:

- Work without Internet connection
- Allow air-gapped version updates

In this configuration:

- The customer manages the hardware and software 24/7
- OVHcloud can manage remotely (via VPN) the software 24/7 as well as the supplier for hardware operations

Platform as a Service SecNumCloud (SNC CloudPlatform)

To be hosted on Platform as a Service SecNumCloud (SNC CloudPlatform), the CloudStore is actually hosted on a Bare Metal Pod hosted in OVHCloud datacenters with SecNumCloud qualification.

The Bare Metal Pod control plane is fully dedicated per customer, and a Landing Zone Manager is made available for SNC users.

2.2.3 CloudStore architecture

Technical concepts

The CloudStore is a K3S cluster and a set of multiple core services (Keycloak, UI, API, etc.). The CloudStore is deployed using FluxCD and GitOps philosophy. This approach provides declarative infrastructure management, version-controlled configuration, and automated reconciliation — ensuring that the actual state of the platform always matches the desired state defined in Git. The cluster runs on a **single node** (no control-plane HA), with **Cilium** as the CNI — so the node is both a single point of failure and the capacity ceiling for one CloudStore.

Services in CloudStore are deployed through GitOps, in two flavours. **Both share the same underlying mechanism:** a Flux **Kustomization** deploys an **OCIRepository** source together with a Flux **Terraform** resource; FluxCD reconciles the Kustomization and OCIRepository, while the tofu-controller reconciles each **Terraform** resource and runs the OpenTofu configuration it points to inside the OCI artifact. The two flavours differ only in **who creates the Kustomizations** and in **how many OpenTofu plans** a service has:

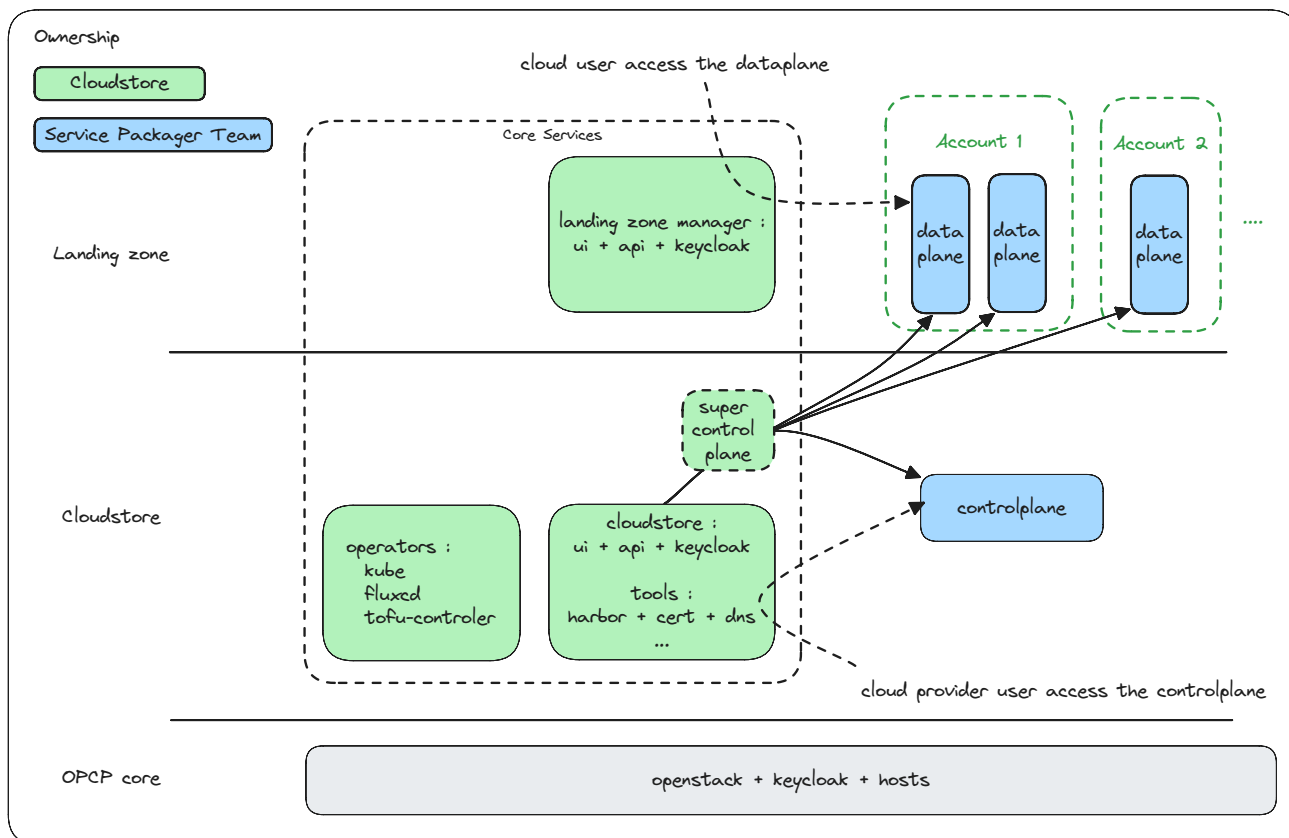
- **Core services** — their Kustomizations are provisioned as part of the CloudStore platform installation. A core service has a **single OpenTofu plan** and no controlplane/dataplane split.
- **Customer services (Cloud App Catalog)** — their Kustomizations are provisioned on demand through the CloudStore API. This lets a service provision its own infrastructure (OpenStack instances, networking, storage) through one standardized, reproducible process, and it spans **two planes** — the controlplane (controller) and the dataplane (tenant) — each backed by its own OpenTofu plan.

Logical architecture overview

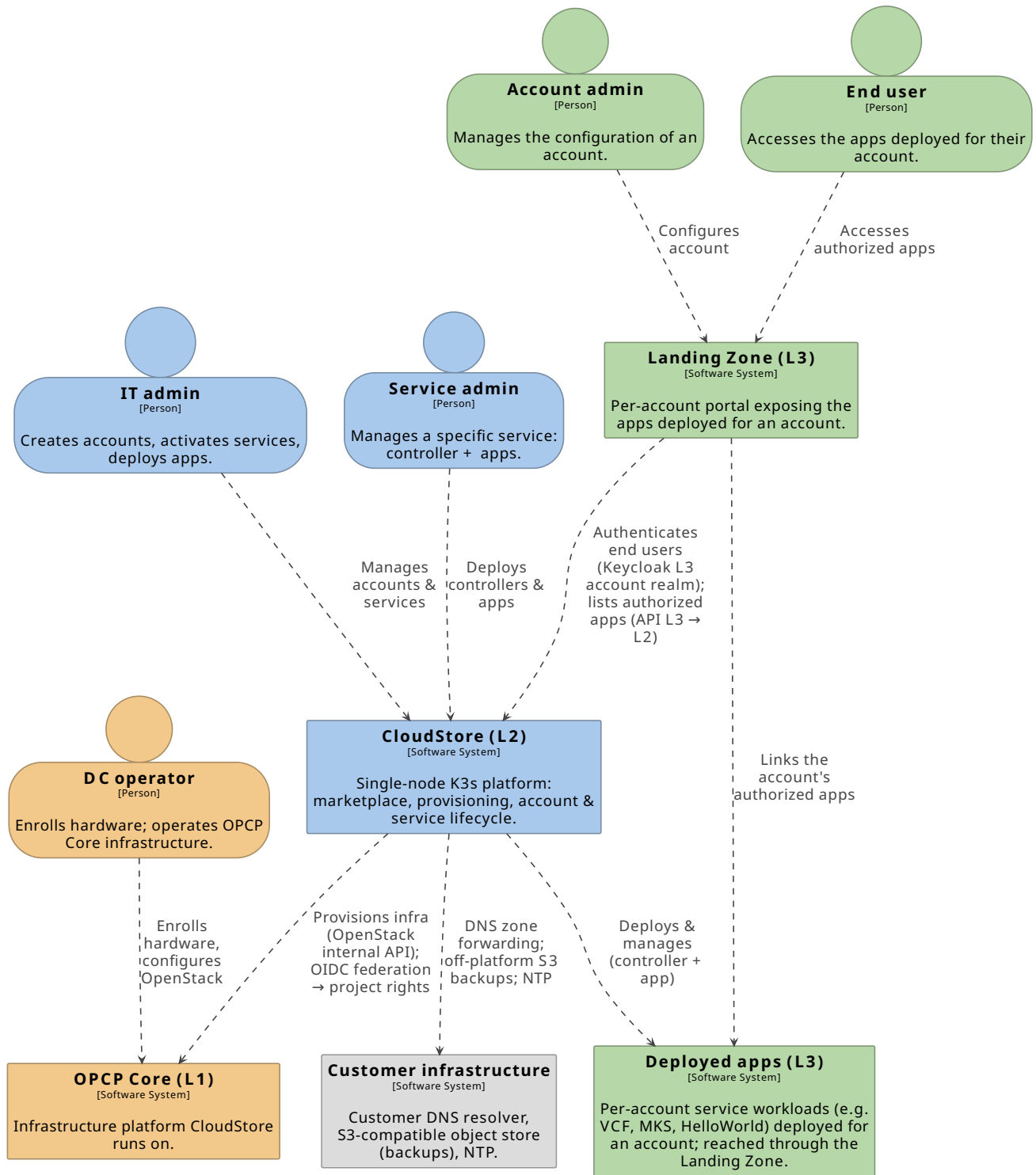
The CloudStore is currently responsible for both the CloudStore Core and the Landing Zone Manager, each operating on a distinct architectural layer. The CloudStore Core handles account provisioning and the deployment of control planes and data planes. The Landing Zone is responsible for displaying authorized applications to end-user clients.

The control plane (aka. a controller) is responsible for managing and orchestrating the data planes (aka. apps).

The Service Packaging teams own the definition, configuration, and maintenance of all components deployed by the control planes and the data planes of their services, ensuring both configuration integrity and proper operational behavior.



System Landscape View L1 / L2 / L3 system landscape



System-context view — see the [architecture overview](#) for the full C1–C3 diagram set.

Core services

Core services are controlplane-only, hidden from the catalog UI, and deployed via GitOps from an OCI artifact. They are grouped by function:

Group	Services
Security & Network	OPA Gatekeeper, cert-manager / trust-manager, PowerDNS, ExternalDNS, Traefik (ingress)
Data & Storage	CNPG (CloudNativePG / PostgreSQL operator), Harbor (OCI registry), Velero
Identity & Access	Keycloak L2, Keycloak L3
Platform Apps	CloudStore API, CloudStore UI, Landing Zone Manager
Observability	Prometheus, Grafana, Loki, Alloy, AlertManager. This core service is temporary and is in a refactoring phase. It will become a new type of service, an enabler service

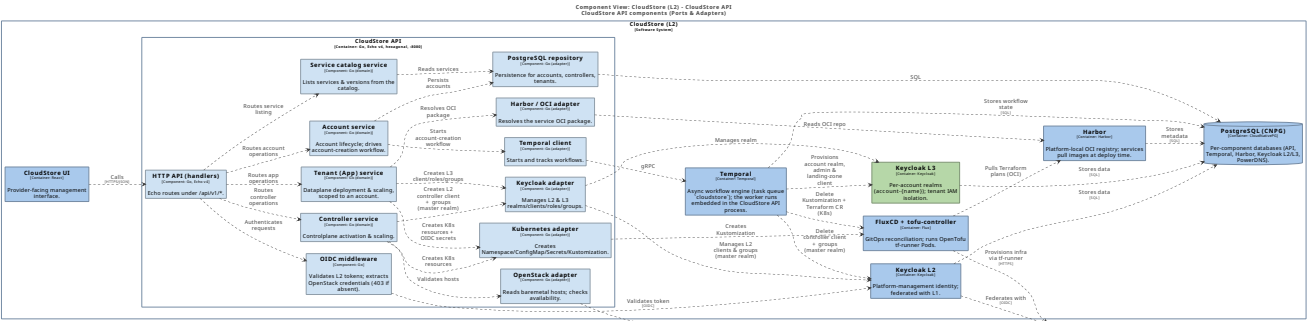
See [Networking](#), [Data & Storage](#), [Security & Compliance](#) and [Observability](#) for how each group is used.

Platform components

The **API**, **UI** and **Landing Zone Manager** are themselves core services (the *Platform Apps* group in the table above); they are broken out individually here because of their central role. The remaining core services are grouped in the last row.

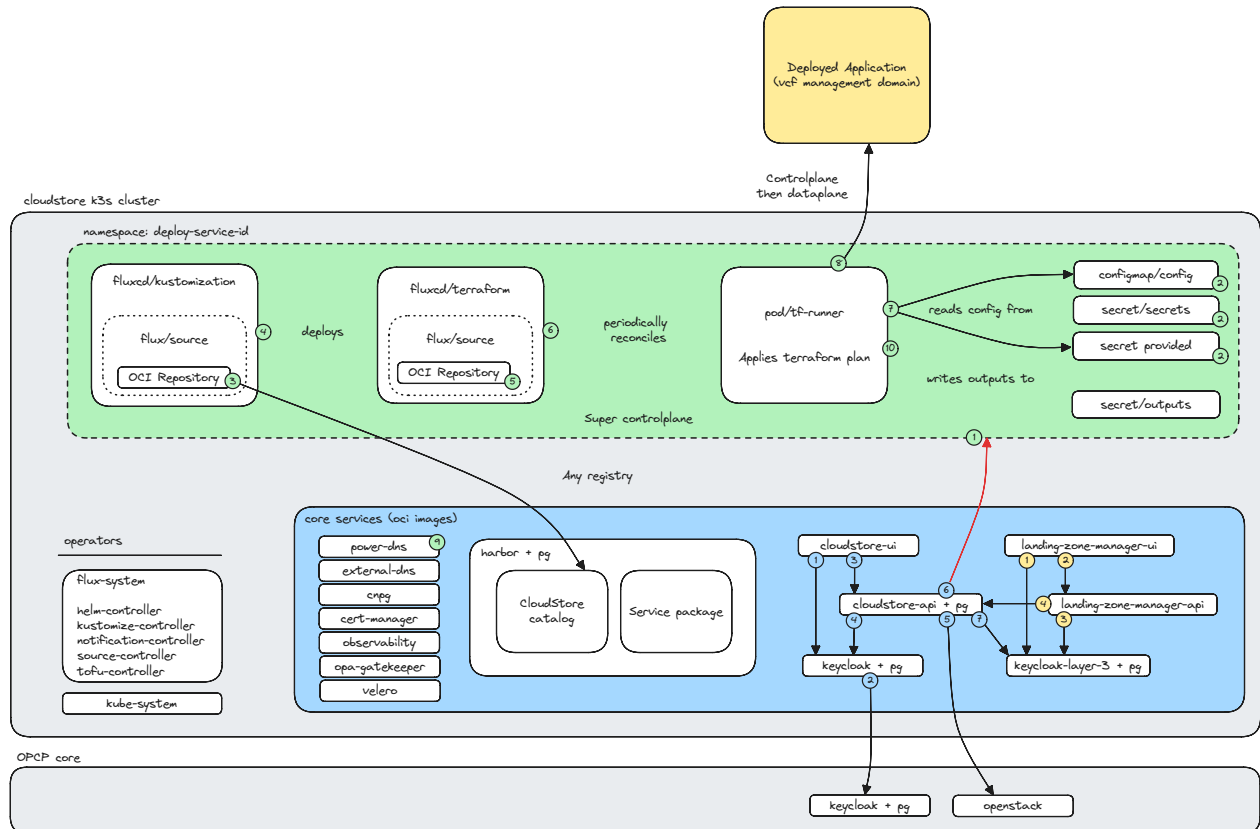
Component	Responsibility	Technology
API	Account provisioning; orchestrates controller/app deployments; reads baremetal hosts	Go, Echo v4, strict hexagonal (Ports & Adapters), port 8080; uses a Temporal workflow engine for async work, backed by its own PostgreSQL database
UI	Provider-facing management interface	React
Landing Zone Manager	Displays authorized apps to end users (L3)	A UI (/) plus a backing API (/api , Go, :8080) behind a single Ingress
Other core services	Keycloak, Harbor, CNPG, DNS, observability, policy	Deployed via GitOps from OCI (see table above)
Controller / app (per service)	The controller manages the lifecycle of apps; apps are per-account workloads	OpenTofu via tofu-controller

The CloudStore API follows a strict hexagonal (Ports & Adapters) design; its internal structure is the C4 component view below.



Deployment architecture

The deployment of a controlplane or dataplane follows a layered orchestration pattern: the API creates Kubernetes resources (Namespace, ConfigMap, Secrets), then a Kustomization takes over asynchronously to pull the service's Terraform plans from an OCI repository and reconcile them through a tf-runner Pod. This design decouples the synchronous API response from the actual infrastructure provisioning, allowing deployments to scale without blocking.



- | | |
|---------------------------------------|---|
| ① ① Authentication request | ① Create a namespace |
| ② Layer 1 Federation | ② Create a ConfigMap + Secrets + secrets-provided |
| ② ③ API call | ③ Use templates retrieved from the catalog to create a customization resource |
| ③ ④ Authentication validation | ④ Resource deployment |
| ⑤ OpenStack project & host management | ⑤ Fetch service configuration |
| ⑥ Super controlplane deployment | ⑥ Runner deployment and reconciliation |
| ⑦ Realm creation | ⑦ Read ConfigMaps and Secrets |
| ④ API call | ⑧ Terraform deployment (controlplanes or dataplanes) |
| | ⑨ DNS record creation |
| | ⑩ Store Terraform outputs in a Secret |

See [Features Reference — Activate a service](#) and [Features Reference — Create an account](#) for the detailed technical specifications.

2.2.5 Roles Overview

Cloudstore team

The CloudStore team is a sub-team of VU.core. It is responsible for core services and provides support to service packagers.

For further information or support, please visit the [Contact Us](#) page.

Service packagers

Service packagers are teams integrating their service in the CloudStore. This packaged services will be available to several OVHCloud offers. Its distribution will be managed by the CloudStore.

Service packager teams will *soon* be able to rely on existing packages (Terraform modules) and have validation tests over hardwares performed on their behalf.

Packaging is documented in the CloudStore Service Packager documentation.

Cloud providers

Cloud providers are OVHcloud customers, making the CloudStore available to their own customers.

Cloud users

Cloud users are Cloud providers' customers.

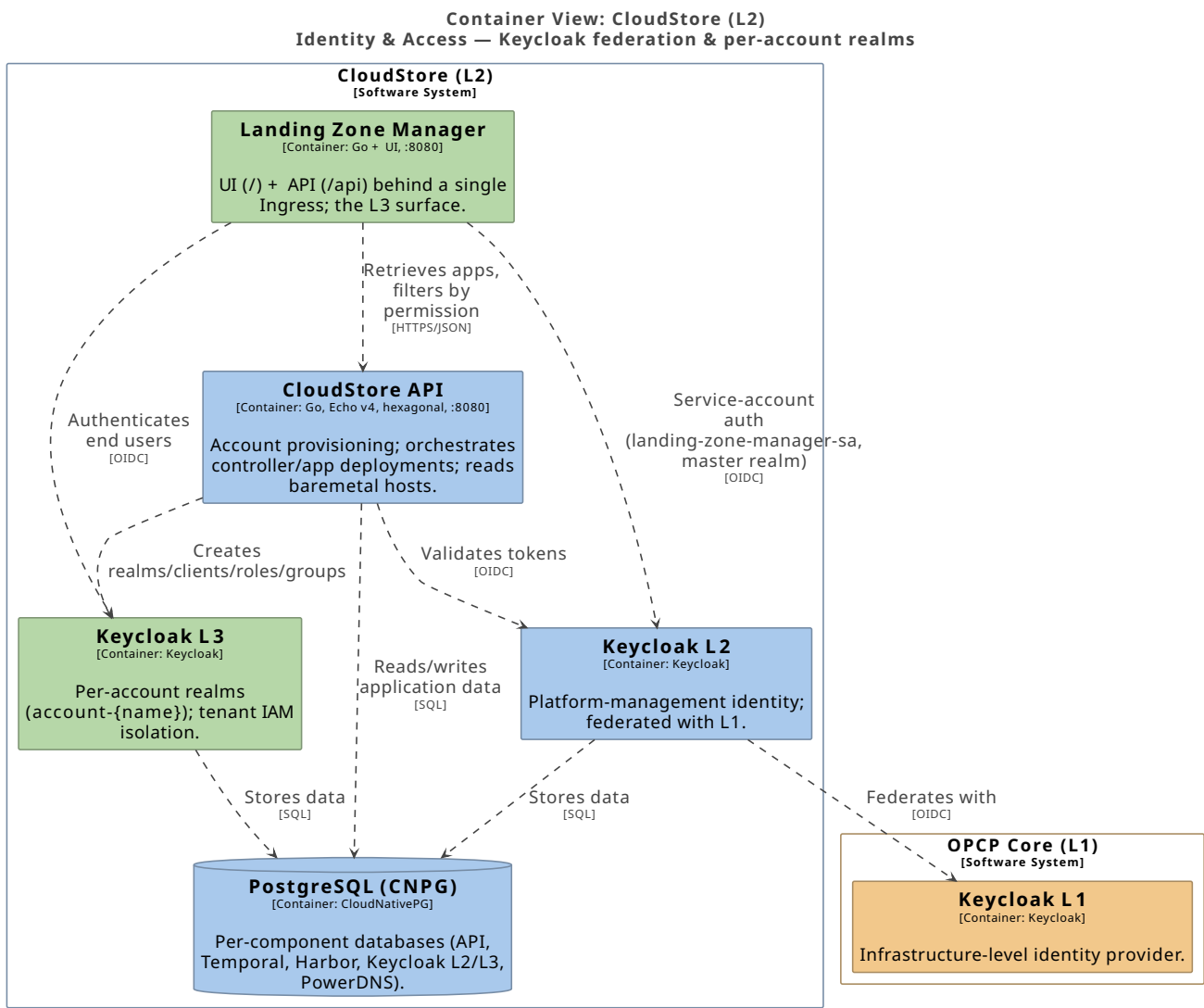
2.3 Identity & access

2.3.1 CloudStore Authentication

Overview

CloudStore's authentication relies on [Keycloak](#), an open source identity and access management solution. The system uses a layered federation model that mirrors the OPCP stack architecture:

Layer	Keycloak instance	Purpose	Users
L1	OPCP Core Keycloak	Infrastructure-level identity	DC operators, Super admins
L2	CloudStore Keycloak	Platform management identity	IT admins, Service admins
L3	Per-account Keycloak realms	End user identity	End users



Identity & access container view — part of the [architecture overview](#) (full C1–C3 diagram set).

Keycloak L2 is federated with L1. Keycloak L3 is an independent instance managed by the CloudStore API.

Keycloak L2 — CloudStore level

The CloudStore has its own dedicated Keycloak instance (L2). This is the identity provider for all platform management operations: accessing the CloudStore UI, managing services, creating accounts, and deploying apps.

Keycloak L2 is **federated with OPCP Core's Keycloak (L1)**. This federation means that users authenticated at the OPCP Core level can access the CloudStore without managing a separate set of credentials. It also ensures that the rights granted on OpenStack projects at L1 are carried through to L2, allowing IT admins to manage infrastructure resources from the CloudStore.

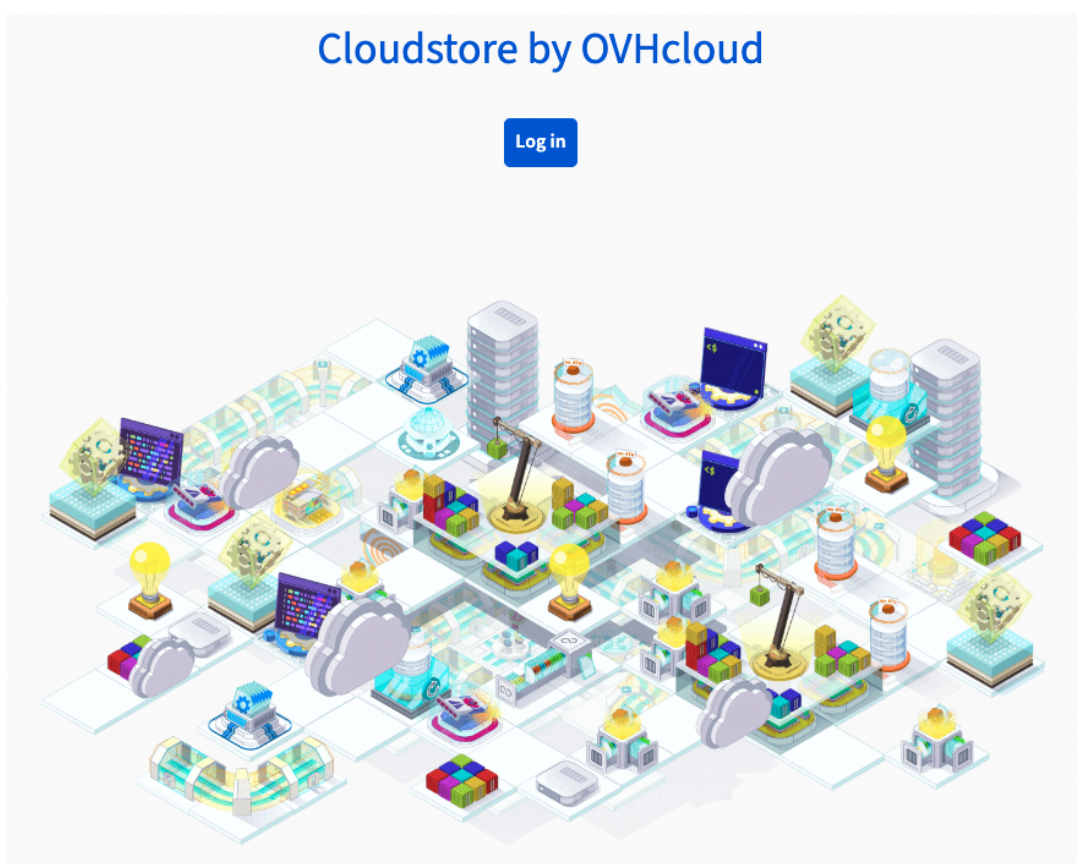
It does not ensure users deleted in Keycloak L1 are also deleted in Keycloak L2 ; deletion of such users is a manual process for now.

Who authenticates through L2

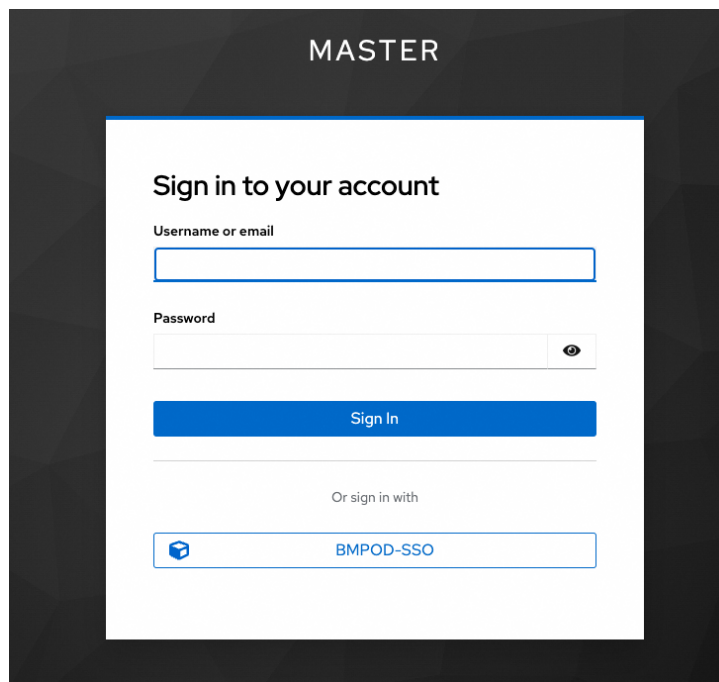
- **IT admins** — manage accounts, services, and platform configuration
- **Service admins** — deploy and manage controllers and apps for their assigned services

Authentication flow

When a user accesses the CloudStore UI, they are presented with a login page:



Clicking the login button redirects to Keycloak L2. The login form offers two options: entering credentials directly in Keycloak L2, or using the **BMPOD-SSO** button to authenticate through the federated OPCP Core Keycloak (L1).



Once authenticated, the UI obtains an OIDC access token that is used for all subsequent API calls. The API validates this token against the Keycloak L2 `master` realm before processing any request, and extracts the caller's OpenStack credentials from the token claims (a request missing them is rejected with HTTP 403).

Role mapping from OPCP Core

When a user authenticates through federation, their roles are inherited from OPCP Core's Keycloak (L1). Assigning the `cloudstore-admins` role to a user in OPCP Core's Keycloak grants them the realm role required to manage the CloudStore Keycloak (L2) — including administering realms, clients, users, and roles.

← → ↻ https://admin.keycloak.demo.bmp.ovhgoldorack.ovh/admin/master/console/#/master/users/e526bf89-8d81-405a-bd5a-54afa34a2d4d/role-mapping

KVM - Demo KVM - Deep KVM - Demo2 KVM - Drannou CloudStore

KEYCLOAK

Keycloak Current realm

Manage realms

Manage

Clients

Client scopes

Realm roles

Users

Groups

Sessions

Events

Configure

Realm settings

Authentication

Identity providers

User federation

Users > User details

pierre-yves.aillet@corp.ovh.com

Details Attributes Credentials **Role mapping** Groups Consents Identity provider links Sessions Events

🔍 Search by name →

☒ Hide inherited roles

Assign role ▼

Unassign

Refresh

☐ Name	Inherited	Description
☐ cloudstore cloudstore-admins	False	—
☐ default-roles-master	False	role_default-roles

See [Features Reference — Access the CloudStore](#) for the detailed sequence diagram.

Keycloak L3 — Landing Zone level

When an [account](#) is created, the CloudStore provisions a dedicated Keycloak realm (named `account-{name}`) within the L3 Keycloak instance. This provides **complete IAM isolation between tenants**: each account has its own users, roles, and groups, and accounts cannot see or interact with each other's identities.

Roles and groups

When an app is deployed for an account, service-specific roles and groups are automatically created in the account's Keycloak realm. For example, deploying a VCF app creates groups like `vcf-<deployment-id>-vcf-users`. Companies can then add users to those groups to grant them access to the deployed app.

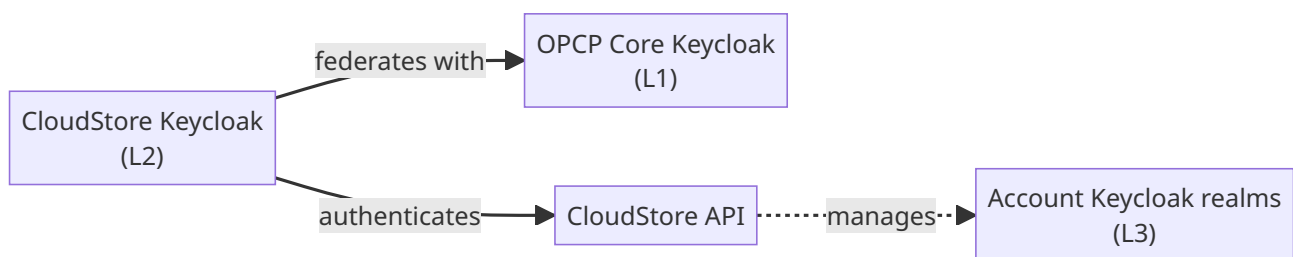
See [Tenancy — Roles and groups](#) for more details.

End user authentication flow

End users authenticate through their account's Keycloak realm (L3) to access the Landing Zone. It is then up to each application to manage authorizations based on the roles defined in its `cloudstore.yaml` and assigned to the groups.

See [Features Reference — Access the Landing Zone](#) for the detailed sequence diagram.

Federation and management model



- **L1 → L2 federation:** Keycloak L2 is federated with OPCP Core's Keycloak, granting CloudStore users the rights they need on OpenStack projects without duplicating credentials.
- **L2 → API authentication:** Access to the CloudStore API is granted through Keycloak L2. Authenticated users (IT admins, service admins) use L2 tokens to call the API.
- **API → L3 management:** The CloudStore API manages Keycloak L3 directly — creating account realms, provisioning OIDC clients, and setting up roles and groups when apps are deployed. L3 is **not** federated with L2; it has its own independent user base per account.

Further reading

- [Glossary — Keycloak](#)
- [Glossary — IAM](#)
- [Tenancy](#)
- [Features — The security model](#)

2.3.2 Service Authorization

Overview

While [authentication](#) establishes *who* a user is, authorization determines *what* they can access. In CloudStore, these two concerns are handled at different levels and by different actors.

Authentication is centralized in Keycloak (L2 for cloud providers, L3 for end users). Authorization, on the other hand, is **service-driven**: each service declares its own permission model, and the platform provisions it automatically at deployment time. There is no central authorization policy that applies across all services.

Why authorization is service-driven

CloudStore is a marketplace that hosts services with fundamentally different access models. A VCF deployment has different roles than a future database service or an observability stack. Imposing a single authorization scheme across all services would either be too restrictive or too generic.

Instead, each service owns its permission model through the `permission` block in its `cloudstore.yaml`. This block declares the groups and client roles the service needs, and CloudStore provisions them in Keycloak upon deployment. A service can define multiple roles to model different levels of access (e.g., read-only users vs. administrators). The service packager decides what makes sense for their service — the platform provisions the roles, but **authorization logic based on those roles must be implemented in the app itself**.

See the [how-to guide](#) for a concrete `cloudstore.yaml` example and the step-by-step setup procedure.

Two levels of authorization

Authorization mirrors the [two-audience split](#) that runs through the entire platform:

Level	Keycloak	Audience	Purpose
L2 (controlplane)	CloudStore Keycloak	Service admins	Control who can manage a service's controller and deploy apps
L3 (app)	Account Keycloak realm	End users	Control who can access a deployed app within an account

This separation exists because the controlplane and the dataplane have different lifecycles, different users, and different security boundaries. A service admin who operates the controller should not automatically have access to every end user's data, and an end user should never see the controlplane.

The `cloudstore.yaml` reflects this split: `control.permission` defines what is created in Keycloak L2, and `data.permission` defines what is created in Keycloak L3 for each account where the app is deployed.

Groups as the authorization primitive

CloudStore uses **groups** — not direct role assignments — as the primary way to grant permissions to users. Roles define *what actions are available* (they are client roles attached to the service's Keycloak client). Groups define *who gets those roles*.

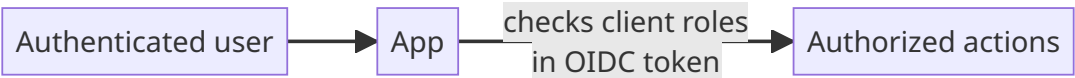
Do not assign roles directly to users

Although Keycloak allows assigning roles directly to users, this is strongly discouraged. Direct role assignments will not be supported by the future group-level authorization feature.

See [Tenancy — Roles and groups](#) for the group model details.

Authorization at runtime

CloudStore's role in authorization stops at provisioning: it creates the client roles and groups in Keycloak, and injects the OIDC secrets so the app can authenticate users. From that point on, **authorization is entirely the app's responsibility**.



When an end user accesses an app, the app receives an OIDC token containing the client roles inherited from the user's group memberships. The app must implement its own authorization logic based on these roles — for example, granting read-only access to users with a `viewer` role and full access to users with an `admin` role.

If a service defines multiple roles, the app is responsible for differentiating between them. The platform does not interpret or enforce role semantics.

The platform guarantees **tenant isolation** (users in one account cannot see another account's apps). Everything beyond that is delegated to the app.

Responsibility boundaries

Concern	Handled by	Level
Is the user who they claim to be?	Keycloak (authentication)	L2 or L3
Can this service admin manage this controller?	Keycloak L2 group membership	Platform (L2)
What can this end user do in the app?	The app itself (client role enforcement)	Application

Further reading

- [Setup service authorization \(how-to\)](#) — step-by-step configuration guide with `cloudstore.yaml` example
- [CloudStore authentication](#)
- [Tenancy — Roles and groups](#)
- [Features — Why two distinct roles](#)
- [Features Reference — Access the Landing Zone](#)
- [Glossary — IAM](#)
- [CloudStore Service Packager documentation](#) — authoring the `permission` block in `cloudstore.yaml`

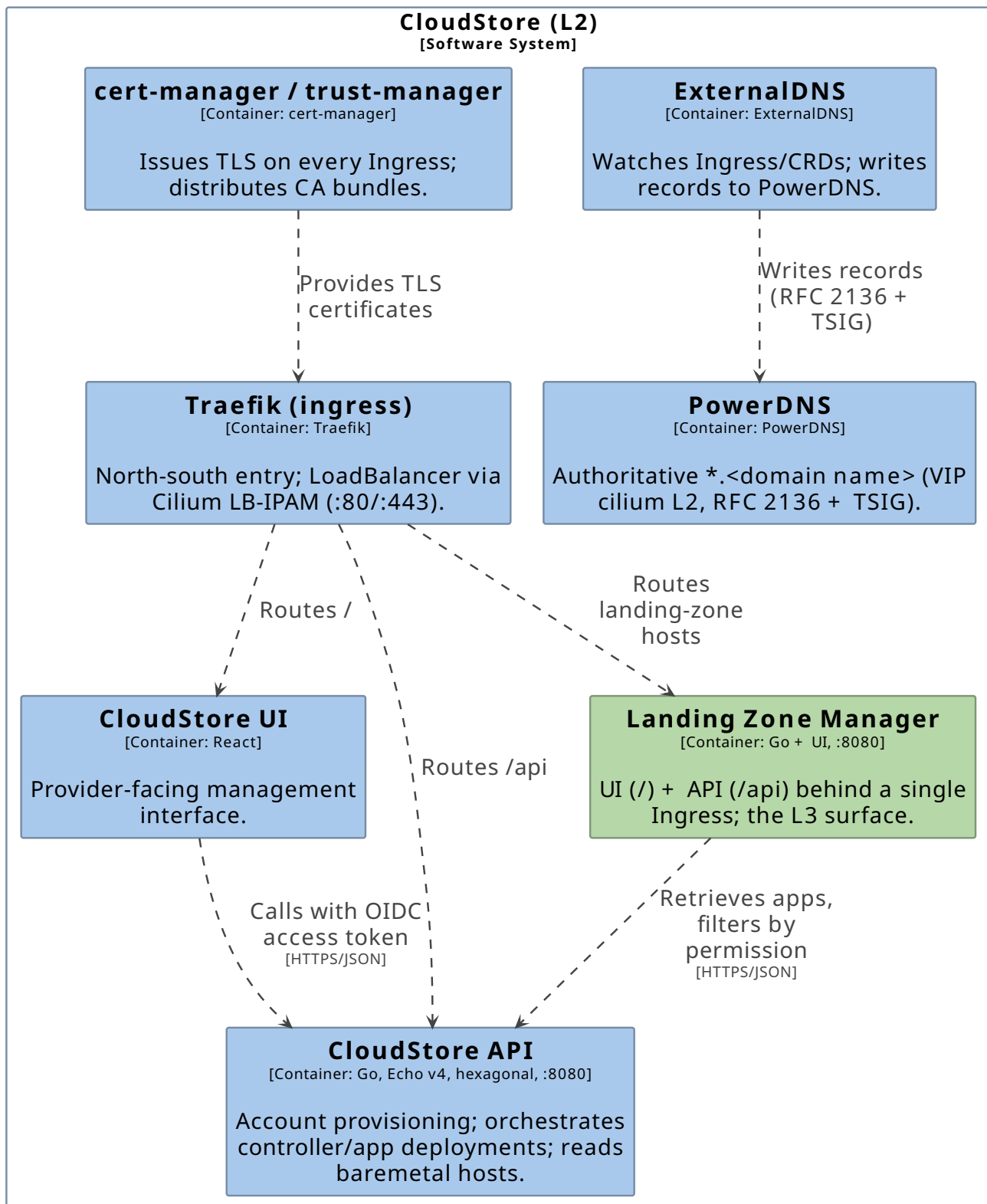
2.4 Platform internals

2.4.1 Networking

This page explains how a CloudStore is connected: the host network model, how traffic enters the platform (ingress), how names are resolved (DNS), how certificates and trust are managed (TLS & PKI), the container network (CNI), the domains and ports in use, and how the platform connects back to the underlying infrastructure (OPCP Core, Layer 1).

CloudStore runs on a **single node**. It is an OpenStack workload deployed on top of OPCP Core and it talks back to OPCP's OpenStack APIs. See [Architecture](#) for the layered (L1 / L2 / L3) model referenced throughout this page.

Container View: CloudStore (L2) Networking & Ingress — north-south entry, TLS & DNS



Networking & ingress container view — part of the [architecture overview](#) (full C1–C3 diagram set).

Host network model

The CloudStore host is attached to **two networks**:

- **admin / management** — administrative access to the host.
- **Customer network** — the network from which the CloudStore is actually reached by customers and end users.

The host security group opens the following ports:

Port	Purpose
22	SSH
6443	Kubernetes (K3s) API server
80	HTTP ingress
443	HTTPS ingress

Ingress and external exposure

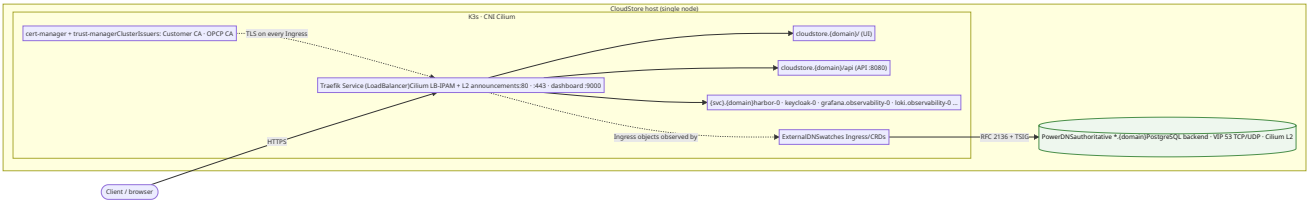
CloudStore uses **Traefik** — the ingress controller bundled with K3s — running in the `kube-system` namespace. HTTPS traffic is routed using standard Kubernetes **Ingress** objects.

Each core or packaged service is exposed as a set of *Deployment* + *Service* + *Ingress* + *RBAC* + *TLS* resources. The Landing Zone Manager UI (served at `/`) and its API (served at `/api`) sit behind a **single Ingress**.

The Traefik **dashboard** is available on container port **9000**. It is not exposed publicly; it is reached by port-forwarding the Traefik pod for administration.

North-south entry

The Traefik (`traefik-customer`) **Service** is of type **LoadBalancer**, realised by **Cilium's LB-IPAM with L2 announcements**: Cilium (rather than the K3s built-in ServiceLB) assigns the service a VIP from the customer pool and advertises it on the customer layer-2 segment. Through a `CiliumL2AnnouncementPolicy`, an elected leader node answers ARP requests for that VIP on its customer-facing interface, so that clients on the customer network can reach the ingress.



Observability

Traefik is monitored: a `PodMonitor` scrapes its metrics, alert rules cover its error rates, and it has a dedicated Grafana dashboard.

DNS

Authoritative resolution — PowerDNS

CloudStore ships **PowerDNS** as the authoritative name server for the `*.<domain name>` zone. It is backed by PostgreSQL and exposed on a **LoadBalancer VIP for port 53 (TCP and UDP)** — assigned by Cilium's **LB-IPAM** and advertised on the customer layer-2 segment through **Cilium L2 announcements**, the same mechanism used for ingress. It accepts **RFC 2136 dynamic updates secured with TSIG**. During setup, the forward and reverse zones, the TSIG keys, and the SOA/NS/A records are created.

PowerDNS is **client-facing**: the customer **forwards a cloudstore dedicated zone (e.g. `<domain name>`) to it**, and CloudStore answers authoritatively. Any split-horizon behaviour lives on the customer resolver side.

Record automation — ExternalDNS

ExternalDNS watches Kubernetes **Ingress** and CRD resources and writes the corresponding records into PowerDNS over **RFC 2136 + TSIG**. The rule of thumb is *ExternalDNS for Ingresses, PowerDNS directly for other resources*.

On a single-node cluster, every hostname (UI, Keycloak, Temporal, deployed apps, ...) resolves to the same **ingress VIP**, announced by the one node — which, being the only node, is always the L2 leader.

Cluster-internal resolution — CoreDNS

CoreDNS provides cluster-internal name resolution, as in any Kubernetes cluster.

Exposed to packaged services

A packaged service can register its own DNS records. The TSIG credentials and the PowerDNS URL are provided to the service as Terraform inputs (`dns_tsig_key`, `dns_tsig_algorithm`, `dns_tsig_secret`, `internal_dns_server`).

TLS and PKI

TLS is issued and renewed automatically by **cert-manager** on **every Ingress**. **trust-manager** distributes CA bundles across the cluster. An ACME **DNS-01** solver validates challenges against PowerDNS.

Three **ClusterIssuers** are configured:

ClusterIssuer	Used for
Customer CA	Internal services (optional)
OPCP CA	Infrastructure-level trust

CA auto-injection

The cluster CA is automatically mounted into workloads by OPA Gatekeeper mutating webhooks, so that pods trust CloudStore-issued certificates without per-service configuration:

- All pods (except those in system namespaces) receive the CA at `/etc/ssl/certs/ca-certificates.crt`.
- Java pods (`app-language: java`) additionally receive `/etc/ssl/java/bundle.jks` together with a `JAVA_TOOL_OPTIONS` setting that points at it.

The CA is also provided to packaged services as the `ca_crt` Terraform input.

Air-gapped: no public ACME

The issuing CA is provided by **OPCP Core (Layer 1)** — there is **no public ACME**. To reach a CloudStore environment, clients must trust that CA (and route through the platform proxy). This is a deliberate property of the air-gapped deployment model.

CNI

Cilium is the Container Network Interface. Its images are bundled into the OPCP core artifacts and it is installed alongside K3s during platform installation.

Domains and ports

The base domain below `<domain name>` is provided as an example, the customer should provide its own DNS zone. Exposition of the CloudStore will then be done on the records shown below. The general URL pattern is:

```
https://{service}.<domain name>{path}
```

Representative service hostnames and in-cluster ports:

Service	Host / path	In-cluster port
CloudStore UI	<code>cloudstore.<domain name>/</code>	—
CloudStore API	<code>cloudstore.<domain name>/api</code>	8080
Landing Zone Manager	<code>landing-zone-manager-0.<domain name></code> (UI / , API /api)	UI 80, API 8080
Keycloak L2 / L3	<code>keycloak-0...</code> / <code>keycloak-layer-3...</code>	—
Harbor	<code>harbor-0.<domain name></code>	—
Grafana / Prometheus / Loki / AlertManager	<code>grafana.observability-0...</code> / <code>prometheus.observability-0...</code> / <code>loki...</code> / <code>alertmanager...</code>	—

Ports summary:

DNS and NTP

CloudStore exposes its own DNS (PowerDNS) on the customer network, and the customer **forwards a zone to it**. CloudStore can in turn **forward non-authoritative queries to the customer/OPCP resolvers**. NTP is provided by the **customer/OPCP** time source.

Air-gapped operation and remote management

In air-gapped mode, every binary is **pre-staged on the host** and GitOps reconciliation reads from a local path — CloudStore **needs no Internet access**. Host interfaces (OOB + customer) are wired at install time, and local accounts, SSH, sudo, and the pre-staged container images are provisioned by the installer. When contracted for managed operation, OVHcloud reaches the platform remotely over a VPN into OPCP Core (or an equivalent private connection). See [Hosting the CloudStore](#).

Current limitations and planned work

What CloudStore networking does not do today

- **No network-level tenant isolation.** There is no [NetworkPolicy](#) or [CiliumNetworkPolicy](#); isolation between tenants is enforced at the IAM realm and Kubernetes namespace levels only (see [Tenancy](#)). Tenant workloads share the node's network paths.
- **No Gateway API, service mesh, or east-west policy.** Ingress uses plain Kubernetes [Ingress](#) objects; there is no service-mesh or Gateway-API-based traffic management.
- **Egress / air-gap is not enforced by CloudStore.** The air-gap is a property of the customer datacenter network. CloudStore assumes no outbound Internet access rather than enforcing it; there is no CloudStore-managed firewall or egress proxy.
- **Tenant (dataplane) networking is not automated in the first release.** Networks, subnets, security groups, and floating IPs must be **pre-created on OPCP**, and the node assigned to the right project, before an app can be deployed.
- **No traffic inspection between layers.** There is currently no inspection between OPCP Core, CloudStore, and the Landing Zone; CloudStore and the Landing Zone Manager share the same network/NIC. Network separation is planned.

Further reading

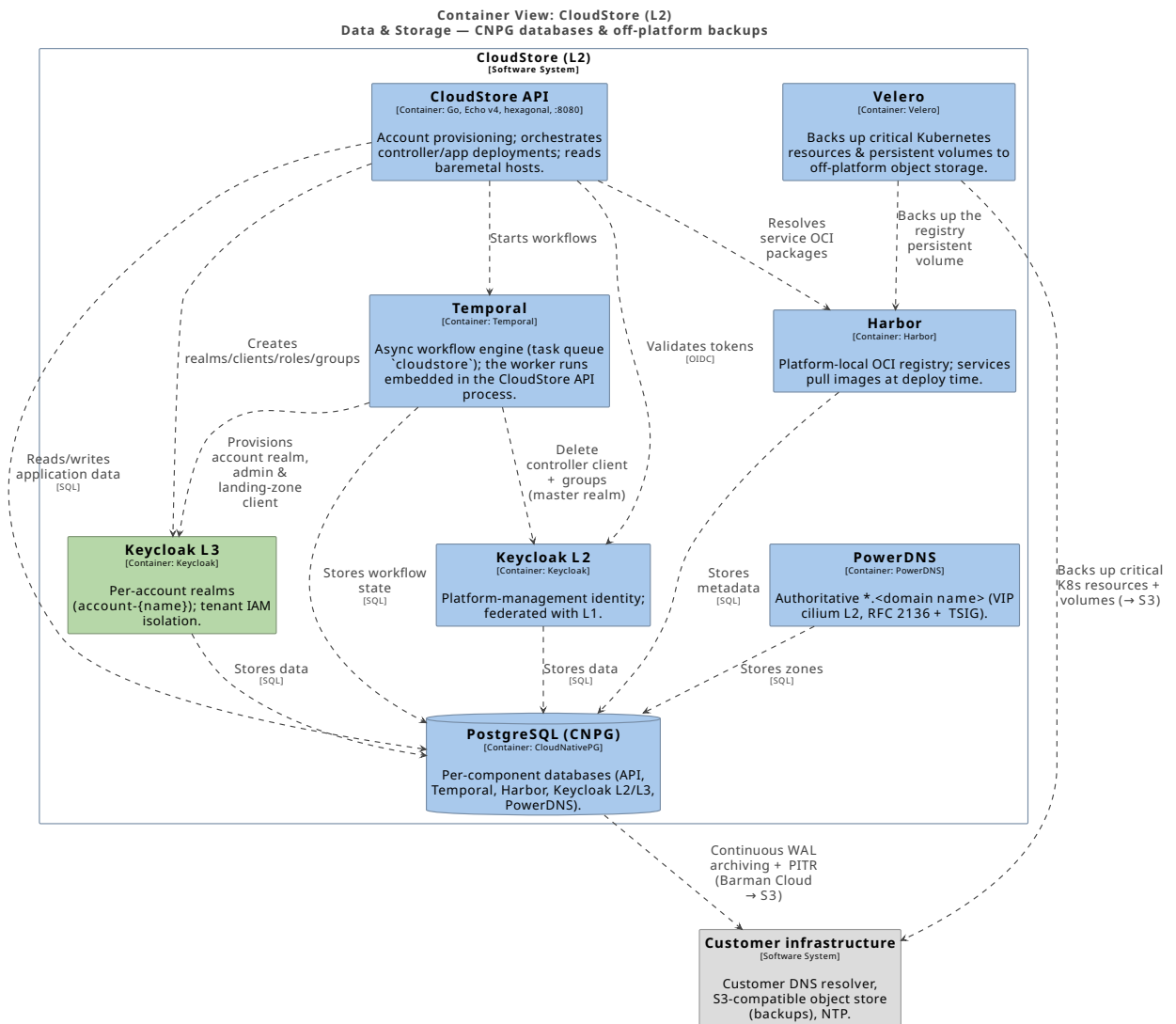
- [Architecture](#)
- [Authentication](#)
- [Hosting the CloudStore](#)
- [Tenancy](#)
- [Services](#)
- [Glossary](#)

2.4.2 Data & Storage

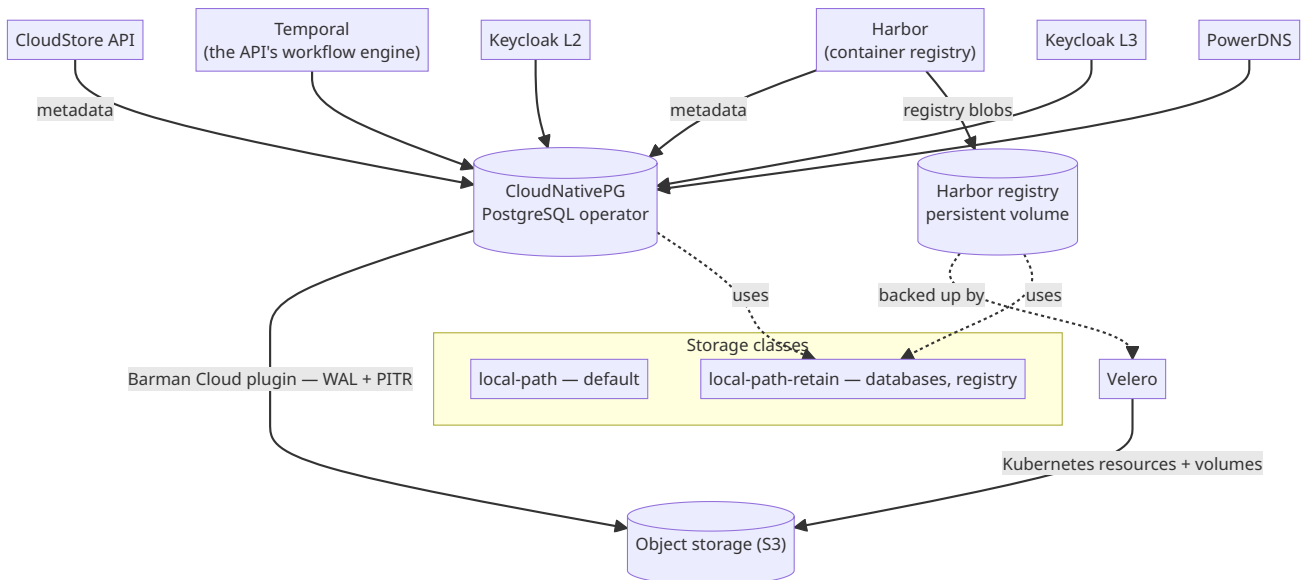
This page describes where CloudStore keeps its data, how that data is persisted on the platform, and how it is backed up and recovered.

Overview

CloudStore stores its application data in **PostgreSQL**, managed on Kubernetes by the **CloudNativePG** (CNPG) operator. Persistent volumes are provisioned through storage classes on the node, and both the databases and the wider set of Kubernetes resources are protected by a layered backup strategy that can target a customer-provided, off-platform object store.



Data & storage container view — part of the [architecture overview](#) (full C1–C3 diagram set).



Application data — PostgreSQL via CloudNativePG

Application state lives in PostgreSQL. Rather than running a single shared database, CloudStore provisions dedicated databases per component, all managed by the CNPG operator. Running PostgreSQL through an operator means database provisioning, high-availability, failover, and backup are handled declaratively as Kubernetes resources.

The following databases are managed on the platform:

Database	Owner / consumer
API	The CloudStore API
Temporal	The workflow engine used by the CloudStore API (owned by the API)
Harbor	The container registry
Keycloak L2	The CloudStore-level identity provider
Keycloak L3	The per-account identity provider
PowerDNS	The DNS service

Temporal

The CloudStore API orchestrates long-running operations (such as account creation and app deployment) with [Temporal](#). Temporal keeps its own workflow state in a dedicated database.

See [Authentication](#) for the roles of the Keycloak L2 and L3 instances.

Storage provisioners

Persistent volumes are backed by two storage classes:

- `local-path` — the default storage class, used for general-purpose persistent volumes.
- `local-path-retain` — used for data that must survive the lifecycle of the workload that created it, namely the **databases** and the **container registry**. Volumes provisioned with this class are retained rather than deleted when their claim is removed, protecting the underlying data against accidental teardown.

Storage is shared across tenants

Storage is provisioned from the node's local disks. There is currently **no per-tenant storage segregation or I/O quota** — tenant workloads share the node's storage. See [Tenancy — Scope of tenant isolation](#) for the current isolation boundaries.

Where state lives

Terraform state and outputs

Infrastructure provisioning is driven by Terraform. Its **state** and its **outputs** are stored as **Kubernetes Secrets** in the same namespace of the Terraform Custom Resource (CR). This keeps the provisioning state co-located with the cluster it describes.

Instance state

The state of a deployed instance is **derived from its Kubernetes Kustomization resource**. The reconciliation of that resource on the cluster is the source of truth for what is actually deployed, which keeps the observed state aligned with the declarative configuration rather than with a separate record that could drift.

Note

Because instance state is derived from Kubernetes resources and Terraform state lives in Kubernetes Secrets, backing up the cluster's Kubernetes resources (see below) is as important as backing up the databases.

Backups

CloudStore combines two complementary backup mechanisms so that both the database contents and the surrounding Kubernetes platform can be recovered.

- **Barman Cloud → object storage (databases)**. The CNPG databases are backed up with the **Barman Cloud** plugin, which continuously archives write-ahead logs (WAL) to an S3-compatible object store. Combined with the **scheduled** backups, this enables **point-in-time recovery (PITR)** — restoring a database to any moment within the retained window rather than only to the last full backup.
- **Velero (Kubernetes resources and volumes)**. [Velero](#) backs up **Kubernetes resources and persistent volumes**, so the cluster's declarative configuration — including the Kustomization resources and Terraform-state Secrets described above — can be restored alongside the data.

Two restore paths coexist, and picking the right one depends on the service: - **Single-database services — PITR**. For a service whose state lives entirely in one PostgreSQL database, PITR restores it cleanly to any moment within the retained window. - **Services with a persistent volume — coordinated restore**. When a service also keeps state on a PV (for example **Harbor**), the database and the volume must be restored to the **exact same instant**: the CNPG/Barman database restore and the Velero PV backup have to line up, otherwise the restored service is inconsistent and non-functional. - **Manual global backups — restore by backup ID**. Global backups triggered manually are **not** replayed through PITR. They are restored from a specific backup by setting `targetImmediate = true` and referencing its `backupID`.

Backup policy

Mechanism	Scope	Schedule	Retention
Velero	Kubernetes resources + persistent volumes	Daily	30 days
CNPG / Barman Cloud	PostgreSQL databases	Continuous WAL archiving + a daily full dump	Enables point-in-time recovery

The **daily full dump** schedule is defined per service.

Customer-provided, off-platform backup target

The backup object store is **provided by the customer and hosted off-platform** — it is not stored on the CloudStore node itself. It only needs to be an S3-compatible endpoint reachable on the customer's network, so backups work **without access to the public internet** and are compatible with **air-gapped** deployments.

Provision the backup target

Backups require a reachable S3-compatible endpoint. If no off-platform object store is configured, WAL archiving and scheduled backups cannot complete, and point-in-time recovery will not be available.

Further reading

- [CloudStore architecture](#)
- [Authentication](#)
- [Tenancy](#)
- [Glossary](#)

2.4.3 Data classification & handling

This page inventories the data a CloudStore platform holds, proposes a sensitivity class for each kind, and states how each class must be handled — where it may be stored, who may read it, whether it may leave the platform, and how long it is kept. It also states plainly which of those handling rules the platform **enforces** today and which it merely **assumes**.

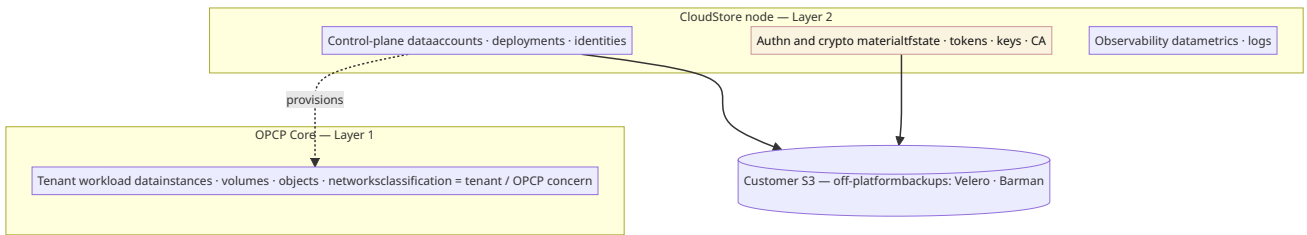
 This scheme is a proposal, not a validated policy

CloudStore has no agreed data-classification policy. The four classes below are a **working proposal**, derived from the data the platform demonstrably holds and from the SecNumCloud egress rules recorded by the Sovereignty group. They have **not** been reviewed or signed off by security or compliance. Treat class assignments as a starting point for that review, not as a control you can cite in a customer security questionnaire.

Scope — three distinct data domains

CloudStore sits between the infrastructure it consumes and the workloads it deploys, so "the data" means three different things. This page covers the first.

Domain	Owner	What it is	Covered here
Platform control-plane data	CloudStore (L2)	Accounts, deployments, identities, secrets, infrastructure state, metrics and logs held by the CloudStore node itself	Yes
Tenant workload data	The tenant, on OPCP (L1)	Contents of instances, volumes, object stores and networks provisioned for an app	No — CloudStore provisions the resources but never reads their contents. Classification of tenant data is an OPCP concern; see Relationship to the OPCP labelling framework
Build & delivery data	OVHcloud, internally	Service packages, registries, CI credentials	Partly — only as it lands on a platform (Harbor contents, package artifacts)



Classification scheme

Four classes, ordered by the harm that disclosure would cause. The decisive test for **C2** is *correlatability*: whether the data can be tied back to an identifiable account, tenant, user or service.

Class	Name	Definition	Examples
C0	Public	Carries no customer-specific information. Publishable without review.	Product documentation, open-source component versions, upstream container images, the <code>cloudstore.yaml</code> schema
C1	Internal	Platform-operational data that reveals nothing about a specific customer, tenant or user.	Service catalogue contents, package versions, Grafana dashboard definitions, platform component inventory
C2	Confidential	Data identifying — or allowing inference about — an account, tenant, end user, deployment or usage pattern. Includes all personal data.	Account and realm names, user names and email addresses, deployment inventories, DNS records, per-service metrics, detailed logs, consumption records
C3	Secret	Authentication and cryptographic material. Disclosure grants access to C2 data or to the infrastructure.	Terraform state and outputs, OIDC client secrets, OpenStack application credentials, DNS TSIG keys, the Keycloak admin password, SOPS decryption keys, CA private keys

i This is not a STANAG / national classification scheme

C0–C3 classify **CloudStore's own platform data** so that it can be handled consistently. They are *not* national or NATO classification levels (UNCLASSIFIED → SECRET), and they carry no legal meaning. Labelling *tenant* data against STANAG 4774 levels is a separate, OPCP-level capability — see [below](#).

Data inventory

Every store a running platform holds, with its proposed class. Retention is as configured today; see [Availability & Disaster Recovery](#) and [Observability](#).

Application databases (PostgreSQL via CNPG)

All six databases run on the node's `local-path-retain` volumes and are backed up to the customer's off-platform S3 by Barman Cloud (continuous WAL + a daily full dump). See [Data & Storage](#).

Data	Store	Class	Why
Accounts, controllers, apps, hosts	CloudStore API DB	C2	The tenant inventory — names, what each account has deployed, on which hosts
Workflow state for account creation and deployments	Temporal DB	C2	Carries the account-admin's full name and email as workflow input — personal data
Provider identities (IT admins, service admins), sessions	Keycloak L2 DB	C2	Personal data
End-user identities, one realm per account, group and role memberships	Keycloak L3 DB	C2	Personal data, per tenant. Realm names (<code>account-{name}</code>) identify customers
Zone records for <code>*.<domain></code>	PowerDNS DB	C2	Hostnames enumerate every deployed service and app
Registry metadata, projects, robot accounts	Harbor DB	C1	Catalogue-level; C3 if robot-account credentials are read

Kubernetes-resident state and secrets

Data	Store	Class	Why
Terraform state and outputs	gzip-encoded k8s <code>Secret</code> in <code>deploy-<service>-<hash></code> namespaces	C3	State routinely embeds credentials, private IPs and generated passwords for the provisioned infrastructure
Per-app OIDC client secrets	k8s <code>Secret</code> , injected at app deploy	C3	Grants a service's identity
OpenStack application credentials (<code>openstack_application_credential_id</code> / <code>_secret</code>)	k8s <code>Secret</code> / Terraform input	C3	Grants provisioning rights on OPCP L1
DNS TSIG credentials (<code>dns_tsig_key</code> , <code>dns_tsig_algorithm</code> , <code>dns_tsig_secret</code>)	k8s <code>Secret</code> / Terraform input	C3	Allows arbitrary record updates in the platform zone
Keycloak admin password	k8s <code>Secret</code> (<code>keycloak-0</code>)	C3	Full control of the identity provider
Issuing CA key material (cert-manager, Customer CA / OPCP CA)	cluster issuers	C3	Allows impersonation of any platform endpoint
SOPS-encrypted values in the GitOps configuration	Git / OCI artifact on the node	C3 as plaintext, C1 as ciphertext	Encrypted at rest in configuration; the age/GPG decryption key is C3 and is scoped per environment
Instance state	Derived from the Kustomization resource, not stored	C2	Reveals what is deployed for whom
Container images and service packages	Harbor registry volume	C1	Becomes C3 if a packager ships credentials inside an image or chart — see the handling rules

Observability and backups

Data	Store	Class	Retention
Metrics (Prometheus)	Node <code>local-path</code> PV	C2 — per-service series are correlatable	Short, bounded by node disk; no off-node store
Logs (Loki, shipped by Alloy)	Node <code>local-path</code> PV	C2 , and C3 whenever a workload logs a token or password	Short, bounded by node disk
Grafana dashboards and configuration	Node PV	C1	Platform lifetime
Velero backups — k8s resources and persistent volumes	Customer-provided S3, off-platform	C3 — contains the tfstate and all other Secrets	Daily backup, 30-day retention
Barman Cloud WAL archive and full dumps	Customer-provided S3, off-platform	C2	Continuous WAL + daily full dump, per-service schedule
Platform global-state backup taken at upgrade	Platform / customer-provided target	C3	One per upgrade

Two C2/C3 stores do not exist yet

Audit trail — there is no centralized record of who did what across the API, Keycloak and the GitOps layer. When it exists it will be **C2** (and **C3** where it records secret access), and it is a prerequisite for any label-based enforcement.

Consumption records — metering is not implemented (no Agora integration). Per-account consumption data will be **C2**; the PRD targets a one-year retention with a cold archive.

Handling rules per class

Rule	C0 Public	C1 Internal	C2 Confidential	C3 Secret
Storage on the platform	Anywhere	Anywhere	Node volumes; databases on <code>local-path-retain</code>	k8s <code>Secret</code> or SOPS-encrypted only — never in a ConfigMap, a Terraform variable file, a Git-tracked plaintext file, or a log line
Encryption at rest	Not required	SED	SED	SED plus application-level encryption (SOPS) wherever the value is at rest outside the cluster
Encryption in transit	Not required	TLS at the platform edge	TLS at the edge; internal hops are not encrypted today	TLS at the edge; treat every in-cluster hop as untrusted
Access	Everyone	Platform operators	Scoped to the owning account: Keycloak realm + namespace	Operators with explicit need; retrieval is a deliberate, traceable act
May leave the platform?	Yes	Yes	Only aggregated, anonymised and non-correlatable — see Egress	No. Never, including in encrypted form, except as part of a backup written to the customer's own S3 target
Backup	Not required	Velero	Barman PITR + Velero	Velero (the backup then inherits C3 — protect the target accordingly)
Appears in logs / metrics / tickets	Yes	Yes	Identifiers only, never contents	Never. A C3 value in a log promotes the whole log store to C3
Deletion	—	With the platform	With the account or deployment	Rotate on suspicion of exposure; note there is no automated rotation today

! Packagers: a service package must not carry C3 material

A service package is an OCI artifact distributed to platforms and stored in Harbor, which the platform classifies as **C1**. Credentials, tokens and keys belong in Terraform *inputs* supplied by CloudStore at deploy time (which land in k8s Secrets), never baked into a chart, an image or a committed `.tfvars`. Harbor runs no scanning that would catch a violation.

Egress: what may leave a platform

A CloudStore platform is designed to operate **air-gapped** — nothing leaves it by default, and it needs no outbound internet access. Only three channels cross the boundary today:

Channel	What crosses	Class	Notes
Backup to S3	Databases, k8s resources, persistent volumes	up to C3	The target is customer-provided and off-platform , reachable on the customer network. It never leaves the customer's control, so this is not an egress in the sovereignty sense — but the encryption and access control of that bucket are the customer's responsibility and are not specified anywhere today .
Remote managed operation	Operator access to the platform	up to C3	Only when OVHcloud is contracted to operate the platform, over a VPN into OPCI Core.
Outbound telemetry	—	—	None. CloudStore sends no telemetry, usage data or diagnostics to OVHcloud or any third party.

The aggregation boundary (SecNumCloud)

For platforms inside a SecNumCloud perimeter, the constraint is not "encrypt it" but "aggregate it". The working interpretation recorded by the Sovereignty group is that only globally aggregated, anonymised, non-correlatable data may leave the qualified perimeter:

Aggregation level	Example	May leave the SNC perimeter
Raw record	A log line, a source IP, an account ID	No
Per account / tenant	CPU consumed by account A	No
Per service	Latency of a named service's API	No
Globally aggregated	Mean CPU across the whole platform	Yes
Aggregated and anonymised	Monthly platform availability	Yes

The governing rule: *if a person or a tool can infer customer information from it, it must not leave*. This applies **even to encrypted data** — detailed logs, security events, resource inventories, keys and configuration are prohibited outside the perimeter regardless of encryption. Conversely, global statistics, consolidated financial or capacity indicators, and averaged anonymised telemetry are permitted.

This interpretation needs formal validation

The table above restates the analysis on the Sovereignty group's *SNC Data Sensitivity* page, which is an **internal interpretation** of the ANSSI SecNumCloud référentiel — not a certified reading of it. Anything built on it (an export, a support bundle, a group-level monitoring feed) needs sign-off from security and compliance before it ships.

What the platform enforces, and what it only assumes

Handling rules are only as good as their enforcement. Today most of the table above is convention.

Enforced by the platform:

- **At-rest encryption** — self-encrypting drives (SED), transparent to workloads, with key handling performed by the drives themselves.
- **Edge TLS** — cert-manager issues and renews a certificate on every Ingress; the CA is provided by OPCP Core (there is no public ACME in an air-gapped platform).
- **Tenant separation of C2 data** — one Keycloak realm and one Kubernetes namespace per account.
- **Secret storage shape** — SOPS encryption for configuration secrets, k8s Secrets for runtime values; OPA Gatekeeper rejects non-compliant resources at admission.

Assumed, not enforced — each of these is a documented gap:

- **In-transit encryption inside the platform.** Internal service-to-service TLS is not enforced, so C2 and C3 data crosses the node in the clear between pods. (OPCP is separately designing an *Encrypted Fabric* for east-west traffic at L1; it does not cover intra-node L2 hops.)
- **Storage segregation.** There is no per-tenant storage segregation and no I/O quota — C2 data from different tenants shares the node's local disks.
- **Network isolation.** No [NetworkPolicy](#) or [CiliumNetworkPolicy](#); tenant workloads share the node's network paths.
- **Accountability.** With no audit logging, there is no record of who read or exported C2/C3 data.
- **Key custody and rotation.** No KMS or HSM is a core service. The SOPS key is per environment and rotated manually; there is no automated rotation for it or for platform credentials.
- **Artifact integrity.** Harbor runs no vulnerability scanning and enforces no signing or provenance, so nothing detects C3 material shipped inside a C1 artifact.
- **Content inspection.** There is no DLP: nothing scans volumes, objects or logs for classified content or for leaked credentials.
- **Labels and mandatory access control.** No resource carries a classification label and no clearance check gates access. See the next section.

Relationship to the OPCP labelling framework

Everything above is *descriptive* — it classifies data that already exists. A separate, **forward-looking OPCP (L1) capability** would make classification *operative*, by labelling resources and enforcing access against a user's clearance.

 **Status: OPCP PRD, draft, not planned**

OPCP — Data Classification Segregation is a **draft PRD with no delivery date**. Nothing described in this section exists today, at L1 or L2. It is summarised here so that the platform-data scheme above is not confused with it.

Its intended shape:

- **Standards** — STANAG 4774 (confidentiality label syntax: level, policy identifier, caveats and releasability), STANAG 4778 (binding labels to objects — soft metadata binding in phase 1, PKI-signed hard binding in phase 2), and ADATP-5636 (NATO Metadata Label Standard, a JSON serialisation for REST APIs, carried as an [X-Classification-Label](#) header).
- **Labelled objects** — Nova instances, Cinder volumes, Neutron networks, Ceph S3 buckets and object prefixes, and Keystone projects. Labels are stored as resource metadata and inherited by derived resources when a volume is cloned or snapshotted or an object copied.
- **Enforcement** — a [security_clearance](#) attribute on the Keycloak user, carried as a claim through the existing Keycloak → Keystone federation, checked by Oslo Policy middleware in the Nova, Cinder, Neutron and Ceph RGW pipelines. Requests below the resource's level are denied with HTTP 403 and logged. The model is Bell–LaPadula (no read up, no write down).

- **CloudStore's part** — displaying labels: a *Classification* column in resource lists and a label assignment dialog, in the CloudStore Admin Panel and the Landing Zone Manager alongside Horizon.
- **Prerequisites CloudStore does not have yet** — an immutable, append-only audit trail for access-decision logging, and a KMS/HSM for the phase-2 signed label binding.

Open questions

These need answers before this page can become policy rather than a proposal:

1. Who owns the classification policy, and who signs off changes to it?
2. Is the C0–C3 scheme the right granularity, and does it need to map formally onto customer schemes (national levels, NATO levels, a prime contractor's C1–C4 baseline)?
3. What are the encryption and access requirements on the **customer-provided backup bucket**, given that Velero backups are C3? Today nothing is specified.
4. Is a support bundle / diagnostic export ever permitted to leave a platform, and if so at what aggregation level?
5. What retention applies to C2 platform data — account and deployment records, identities — after an account is deleted?
6. Which of the enforcement gaps above are blocking for SecNumCloud qualification, and in what order should they be closed?

Further reading

- [Data & Storage](#) — where each store physically lives, and the backup mechanisms
- [Security & Compliance](#) — secrets, policy enforcement, PKI, and the security gap list
- [Tenancy](#) — the isolation boundaries that scope C2 data
- [Availability & Disaster Recovery](#) — backup policy and retention
- [Observability](#) — what metrics and logs are collected and how long they are kept
- [Roadmap & Maturity](#) — the maturity of every capability referenced here

2.4.4 Security & Compliance

This page explains how CloudStore protects tenants, secrets, and data, how trust is established inside an air-gapped platform, and where the platform stands today with respect to SecNumCloud. It also states honestly which security capabilities are not yet in place.

Identity and access isolation

CloudStore relies on [Keycloak](#) for identity and access management. Each account is provisioned with its **own dedicated Keycloak realm**, so accounts cannot see or interact with each other's users, roles, or groups. This gives strong tenant isolation at the IAM layer by default.

Platform management identity (IT admins, service admins) and end-user identity are handled by separate Keycloak instances, following the layered federation model described in [Authentication](#). Authorization for services is covered in [Service authorization](#), and the scope of tenant isolation is detailed in [Tenancy](#).

Secret management

Secrets are managed declaratively as part of the GitOps configuration:

- **Encrypted at rest in Git.** Secrets committed to the GitOps configuration are encrypted with [SOPS](#), so no plaintext secret material lives in the configuration repository. SOPS-encrypted values are decrypted in-cluster at reconciliation time.
- **Infrastructure outputs become Kubernetes Secrets.** Values produced when provisioning infrastructure (Terraform outputs) are materialized as Kubernetes [Secret](#) objects for the services that consume them.
- **Platform service credentials.** Sensitive credentials such as the Keycloak admin password are stored as Kubernetes Secrets rather than in configuration files.

No dedicated secret-vault core service

CloudStore does **not** ship a dedicated HashiCorp Vault (or equivalent) as a core platform service. Secret confidentiality relies on SOPS encryption in Git combined with Kubernetes Secret storage.

SOPS keys and rotation

The SOPS decryption key is **scoped per environment**: each environment has its own key, so compromising one environment's key does not expose another's secrets.

No automated key rotation

There is currently **no automated rotation procedure** for SOPS keys. Key rotation is a manual operation. Broader secret-rotation procedures beyond routine credential renewal are not yet documented.

Policy enforcement

Cluster-level policy is enforced with [OPA Gatekeeper](#), which runs as a core service. Gatekeeper evaluates admission requests against the platform's policies, letting the platform reject non-compliant Kubernetes resources before they are created.

Encryption at rest

Data at rest is protected by **self-encrypting drives (SED)** at the hardware layer. Encryption and key handling are performed by the drives themselves, transparently to the workloads running on the platform.

PKI and TLS trust

CloudStore issues certificates internally through cert-manager. Because platforms run air-gapped, the issuing **Certificate Authority** is **provided by the underlying platform core** rather than by a public certificate authority.

No public ACME

There is no public ACME issuer (such as Let's Encrypt) in an air-gapped platform. Trust is anchored on the platform's own issuing CA, which must be trusted by clients that connect to platform endpoints.

Air-gap operation

CloudStore platforms are designed to operate **without outbound internet access**. All artifacts a platform needs — container images, OS packages, VM/OS images, and object-storage content — are served from services local to the platform (a local container registry, package mirror, image service, and object store). Nothing is pulled from the public internet at deploy time.

This constraint shapes many of the security decisions above (private PKI, no public ACME, locally hosted artifacts) and is closely tied to how dependencies are managed and delivered.

Current gaps

The following capabilities are **not yet implemented**. They are listed here so the platform's real security posture is clear, and several are prerequisites for a full SecNumCloud story.

Audit logging is not implemented yet

There is no centralized, tamper-evident record of who did what across the API, Keycloak, and the GitOps reconciliation layer. Centralized audit logging is required for SecNumCloud and is planned.

Internal service-to-service TLS is not enforced

Confidentiality today rests on at-rest encryption (SED) and TLS at the platform edge. **In-transit encryption between internal services is not enforced**. Traffic that stays inside the platform is not guaranteed to be mutually authenticated or encrypted.

No image supply-chain scanning or signing

The platform-local container registry does **not** run vulnerability scanning, and there is **no image signing or provenance verification** (no cosign / Notation enforcement). Images are trusted by virtue of being present in the local registry, not by cryptographic attestation.

No canonical threat model

There is no single, agreed threat model or trust-boundary document for CloudStore. Security reasoning is currently distributed across component documentation rather than consolidated.

No CloudStore-specific SecNumCloud control mapping

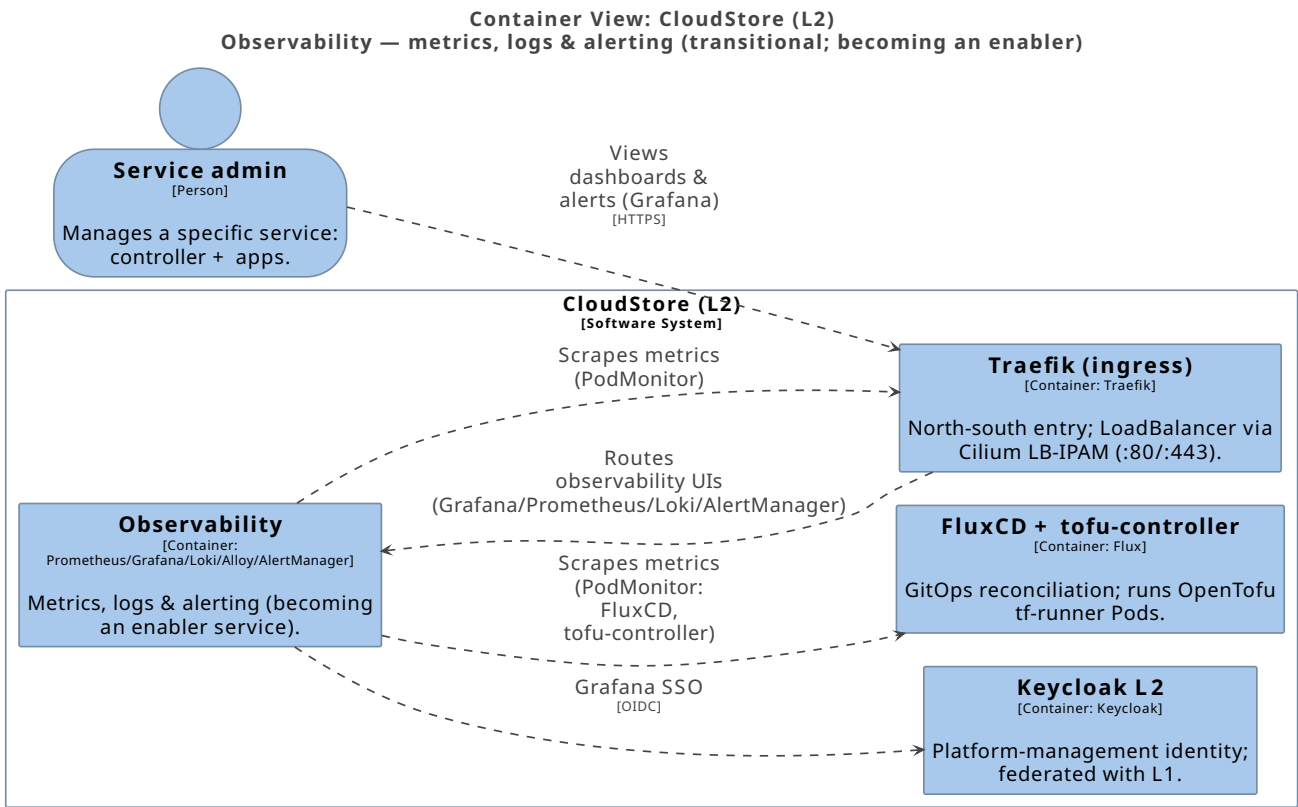
Beyond running on a SecNumCloud-qualified Bare Metal Pod, there is no CloudStore-specific mapping of platform features to individual SecNumCloud controls. Producing that mapping is future work.

Further reading

- [Authentication](#)
- [Service authorization](#)
- [Tenancy](#)
- [Hosting the CloudStore](#)

2.4.5 Observability

CloudStore ships a self-contained observability stack so that operators can see what the platform is doing: metrics, logs, dashboards, and alerting rules are deployed alongside the platform itself. This page explains what is collected, how it is stored, and where the current boundaries are.

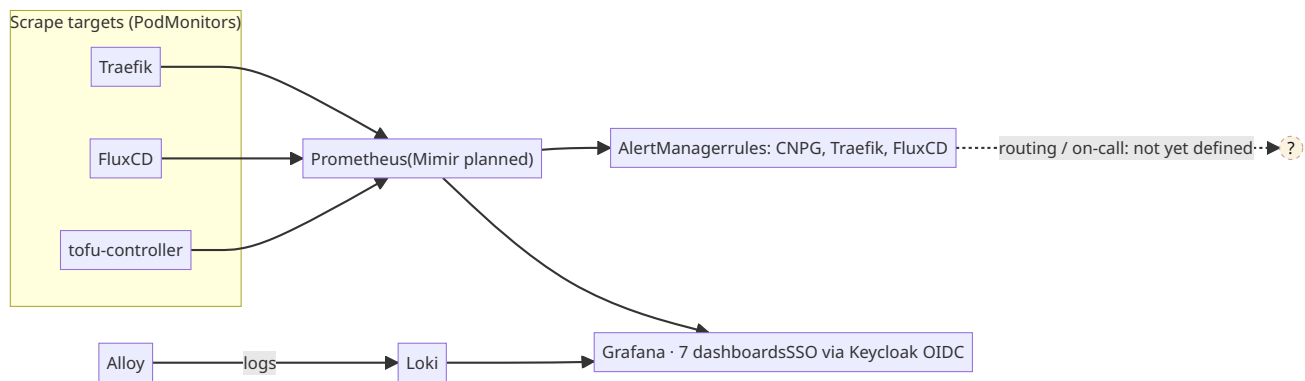


Observability container view — part of the [architecture overview](#) (full C1–C3 diagram set).

Overview

The stack is built from the usual open-source components:

Concern	Component	Notes
Metrics	Prometheus + AlertManager	Prometheus is planned to be replaced by Mimir
Logs	Loki, collected by Alloy	
Dashboards	Grafana	SSO via Keycloak OIDC; 7 dashboards shipped



Metrics

Metrics are collected by **Prometheus**, with **AlertManager** handling alert evaluation. Prometheus scrapes a set of platform components and stores the resulting time series locally.

i Planned migration to Mimir

Prometheus is documented as **planned to be replaced by Mimir**. Mimir would provide horizontally scalable, long-term metrics storage, addressing the retention limits described [below](#). Until that migration happens, metrics remain on Prometheus.

Scrape targets

Scraping is configured declaratively with **PodMonitors**. Today the following platform components are scraped:

- **Traefik** — the ingress controller
- **FluxCD** — GitOps reconciliation
- **tofu-controller** — infrastructure provisioning

Alert rules

AlertManager evaluates alert rules covering:

- **CNPG** — the CloudNativePG PostgreSQL operator
- **Traefik**
- **FluxCD**

⚠ Alert routing and on-call are not yet defined

Alert rules fire into AlertManager, but the **routing configuration and on-call escalation are not yet defined**. In practice this means alerts are evaluated but there is no established notification path (paging, chat, email) or on-call rotation wired up yet. This is a known gap and is planned work.

Logs

Logs are shipped by **Alloy**, which collects pod and platform logs and forwards them to **Loki**. Loki indexes and stores the log streams, and they are queryable from Grafana alongside metrics.

Dashboards

Grafana is the single pane of glass for the platform. It reads from both Prometheus (metrics) and Loki (logs) so operators can correlate the two in one place.

- **Single sign-on** is handled through **Keycloak OIDC** — operators log in with their platform identity rather than a Grafana-local account. See [Authentication](#) for how Keycloak is layered across the platform.
- **7 dashboards** are shipped out of the box, covering the platform components that are scraped.

Retention and storage

Both Prometheus (metrics) and Loki (logs) persist their data on the node's `local-path` persistent volume.

Short retention, bounded by node disk

Because storage lives on the node's local disk, **retention is short and bounded by the available disk space** on that node. There is **no long-term or off-node metrics/logs store yet** — historical data is not preserved beyond what the local volume can hold. This is consistent with, and part of the motivation for, the planned migration to Mimir for metrics.

Operators who need durable, long-term observability data should treat the current stack as short-window and not rely on it for historical analysis.

Current gaps

To be explicit about what the stack does **not** cover today:

- **No distributed tracing.** Only metrics and logs are collected; there is no request-level tracing across components.
- **No long-term / off-node storage.** Metrics and logs are retained only as long as the node's local volume allows (see [Retention and storage](#)).
- **AlertManager routing / on-call not yet defined.** Alerts are evaluated but not yet routed to a notification channel or on-call rotation.

Further reading

- [CloudStore architecture](#)
- [Authentication](#)
- [Glossary](#)

2.5 Packaging & delivery

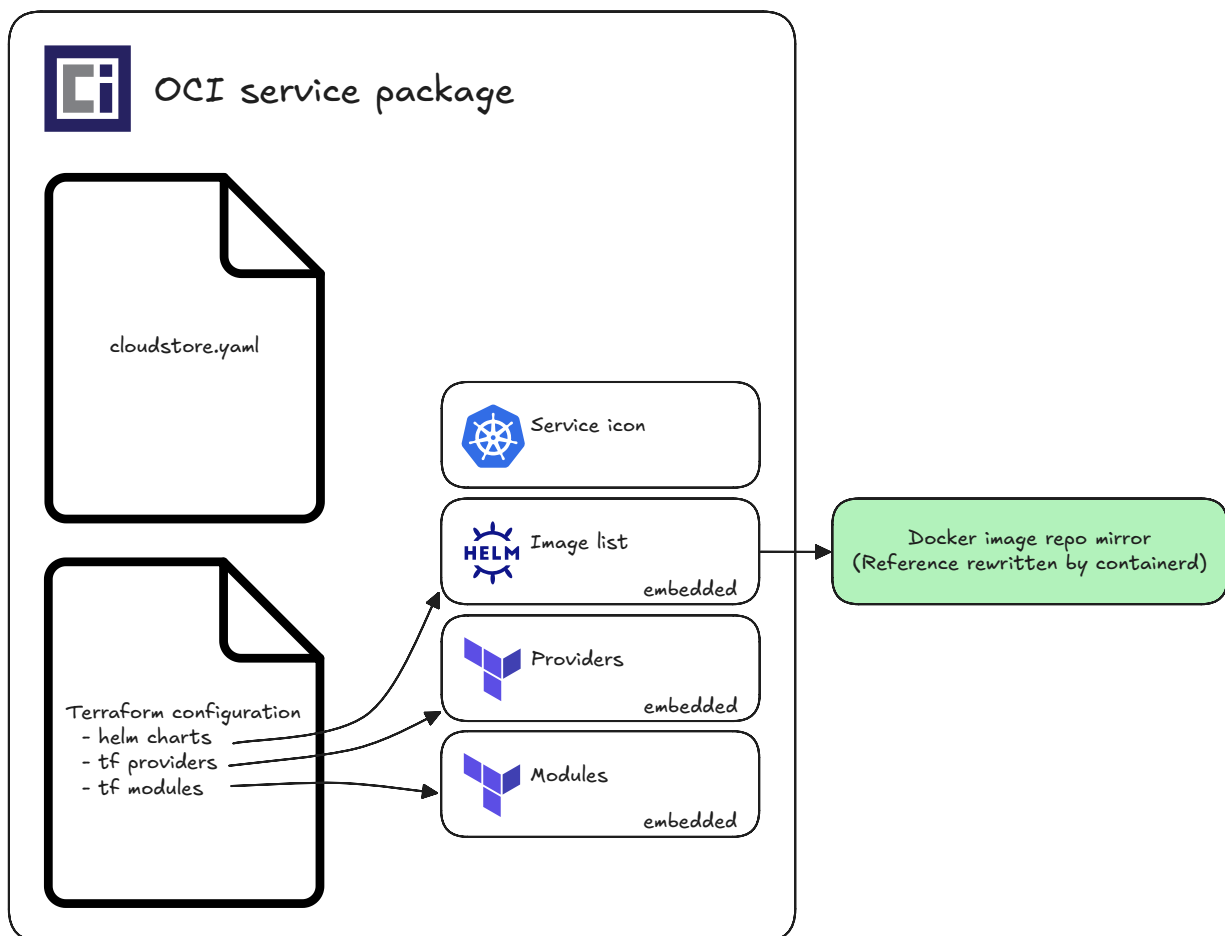
2.5.1 What is a CloudStore service package ?

A service package is an OCI image included in the CloudStore to deploy one or more instances of control planes and one or more data planes. Multiple versions of a service can be integrated in the CloudStore.

Structure of the package

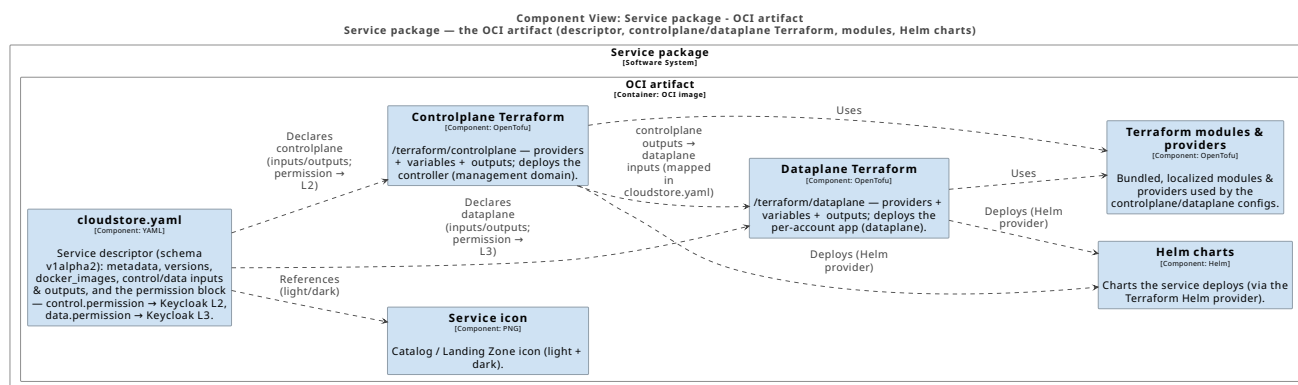
A service package is an OCI artefact which embeds:

- The cloudstore.yaml descriptor of the service
- The service icon
- The Terraform configuration
- The Terraform modules and providers used by the configuration
- The Helm Charts used by the service



What a service package OCI artifact contains.

The same artifact seen as a C4 component view — how the parts relate rather than how they are laid out:



Service-package component view — part of the [architecture overview](#) (full C1–C3 diagram set).

In the OCI artefact, the Terraform configuration is split in 2 directories:

- `/terraform/controlplane` : the Terraform configuration used to deploy the control plane of the service (aka the controller)
- `/terraform/dataplane` : the Terraform configuration used to deploy the data plane of the service (aka the app)

Each directory has its own set of:

- Providers depending on the deployment target
- Variables used to customize the deployment

The variables come from various sources:

- User provided: A "user" is a person (cloud provider) who has access to the CloudStore UI and is able to deploy both the control plane and the data plane
- CloudStore provided: through the API, the CloudStore is able to provide itself some of the variables (service name, deployment ID, etc.)
- Controlplane provided (only for the dataplanes): inputs variables of the data plane can be inferred directly from outputs variables from the control plane, as long as it is specified in the `cloudstore.yaml` file

Versioning & publishing

A service package is a **versioned OCI artifact**. Packages follow **Semantic Versioning 2.0** (driven by commitizen / conventional commits), and the `make generate_package` pipeline (`tf_test` → `build_oci` → `push_oci` → `mirror_docker_images`) builds and publishes the package to a registry. The `cloudstore.yaml` descriptor carries its own Kubernetes-style **schema version** (`version: v1alpha2`), which is independent of the service's semver.

Only Terraform-based services are integrable — CloudStore can only integrate services packaged as a Terraform (OpenTofu) plan.

Air-gapped delivery (metapackage)

To make a service available inside an **air-gapped** platform, a package can be bundled together with all of its external dependencies (OCI image, Docker images, Glance VM images, Debian packages, S3 objects) into a self-contained **metapackage** (`tar.gz`), imported offline.

Under design

The metapackage mechanism is under active design and not yet available. Today, Docker images declared in `cloudstore.yaml` are automatically mirrored into the platform registry during packaging; the other dependency types await the metapackage tooling. See [Package delivery to customer platforms](#).

Enablers for packaging process

The [CloudStore team](#) provides resources to assist service packager teams in packaging their services. The authoritative reference for the packaging toolchain is the CloudStore Service Packager documentation.

Further reading

- [CloudStore Service Packager documentation](#) — authoritative reference for the packaging tool
- [Package delivery to customer platforms](#)

2.5.2 Package delivery to customer platforms

Once a [service package](#) has been built and published as a versioned OCI artifact, it still has to reach the platform where it will be deployed. On customer platforms this cannot rely on a network link back to the central publishing registry, so packages are delivered through an offline mechanism.

On a customer platform

A customer platform is typically air-gapped and has **no replication link** to the central publishing registry. Packages therefore reach it through two different channels depending on the kind of service:

- **Core-service images** are provisioned at **install time**, when the platform is first set up.
- **Cloud-app images** are brought in **after installation**, through an offline import mechanism called the **metapackage** (see below).

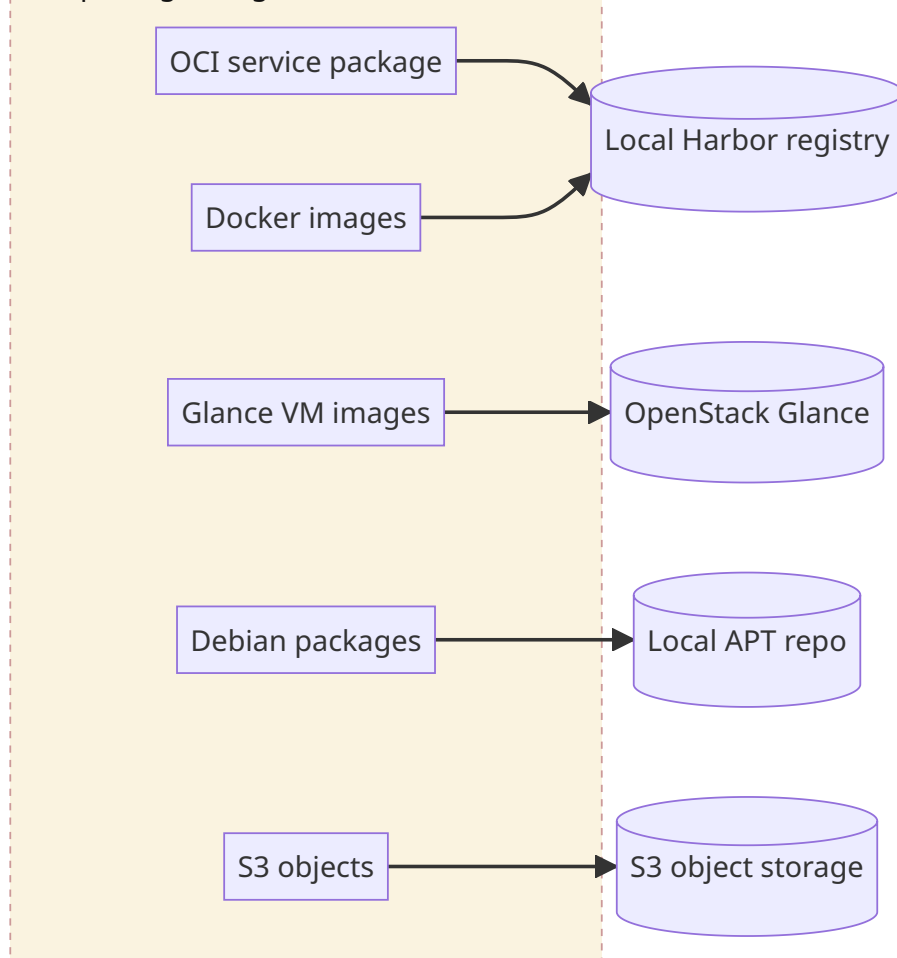
The metapackage (air-gap delivery)

A **metapackage** is a self-contained `tar.gz` archive that bundles a single service package together with **all of its external dependencies**, so that the service can be made available for deployment inside an air-gapped platform with no outbound network access. Installing a metapackage does not deploy the service; it makes the service and everything it needs *available* on the platform.

The relationship is strict: **one service, one version, one metapackage**. Each dependency type in the bundle is imported into the appropriate platform-local store:

Dependency type	Destination
OCI image (the service package itself)	Local Harbor registry
Docker images	Local Harbor registry
Glance images (VM images)	OpenStack Glance
Debian packages	Local APT repository
S3 objects	S3-compatible object storage

metapackage (tar.gz) — 1 service / 1 version



⚠ Under active design

The metapackage mechanism is **under active design and not yet available**. Today, only part of the picture works: the service's OCI image and its declared Docker images are mirrored into Harbor during packaging. The remaining dependency types — Glance VM images, Debian packages, and S3 objects — await the metapackage tooling.

3 How-to Guides

3.1 How-to Guides

Task-oriented, step-by-step guides for people **building and operating services on the CloudStore**. Each guide assumes you already understand the concepts covered in the [Explanation](#) section.

[Setup service authorization](#)

Grant service admin access to the controlplane (L2) and end-user access to a deployed app (L3).

3.2 Setup service authorization

This guide walks through the steps to configure authorization for a service at both levels: granting service admin access to the controlplane (L2) and granting end user access to a deployed app (L3).

The groups and roles provisioned by CloudStore are defined in the service's `cloudstore.yaml`. For example, the VCF service defines:

```
control:
  # ...
  permission:
    groups:
      - name: vcf-admins
        roles:
          - vcf-federation

data:
  # ...
  permission:
    groups:
      - name: vcf-users
        roles:
          - vcf-federation
```

The `control.permission` block drives what is created in Keycloak L2 (controlplane), and the `data.permission` block drives what is created in Keycloak L3 (app). See [Service authorization](#) for details.

3.2.1 L2 — Controlplane access (service admins)

Prerequisites

- A controller has been [deployed](#)
- Access to Keycloak L2 as an IT admin

1. Verify the provisioned resources

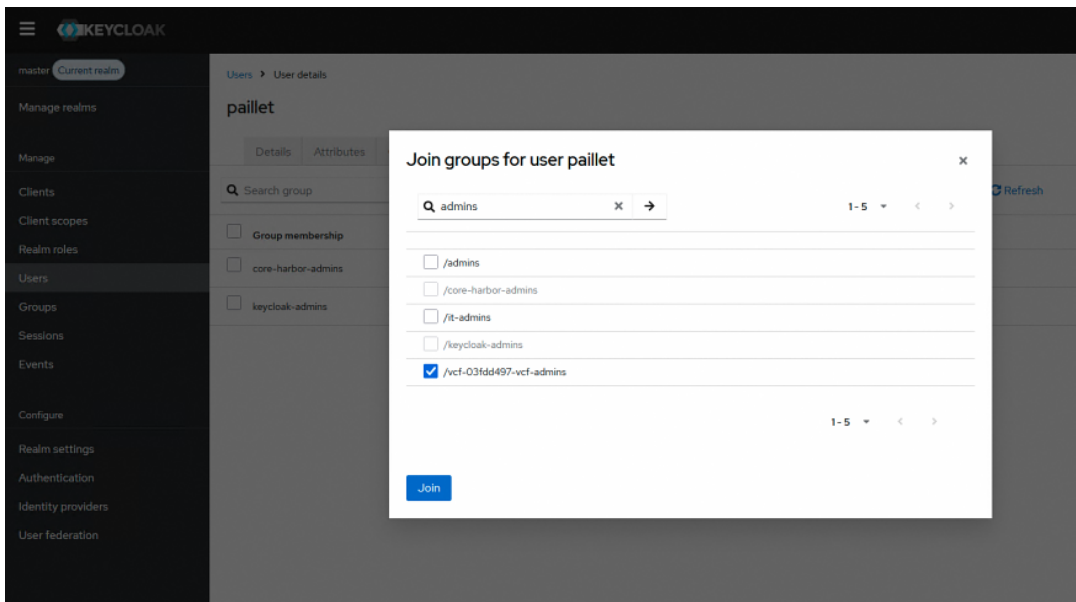
When a controller is deployed, CloudStore creates service-specific groups in Keycloak L2. Verify they exist:

1. Log into the Keycloak L2 admin console
2. Navigate to **Groups** and confirm the service-specific admin group exists (e.g., `vcf-<deployment-id>-vcf-admins`)

2. Assign service admins to the group

Grant a user service admin access to the controlplane:

1. Navigate to **Users** in the Keycloak L2 admin console
2. Select the user
3. Go to the **Groups** tab
4. Click **Join Group** and select the service admin group



The user will now be able to manage the controlplane and deploy apps for this service.

3. Remove access

To revoke a service admin's access:

1. Navigate to **Users** → select the user → **Groups** tab
2. Remove the user from the service admin group

3.2.2 L3 — App access (end users)

Prerequisites

- An account has been [created](#)
- An app has been [deployed](#) for that account
- Access to the account's Keycloak realm (L3) as an account admin

1. Verify the provisioned resources

When an app is deployed, CloudStore automatically creates roles, groups, and a client-id in the account's Keycloak realm. Verify they exist:

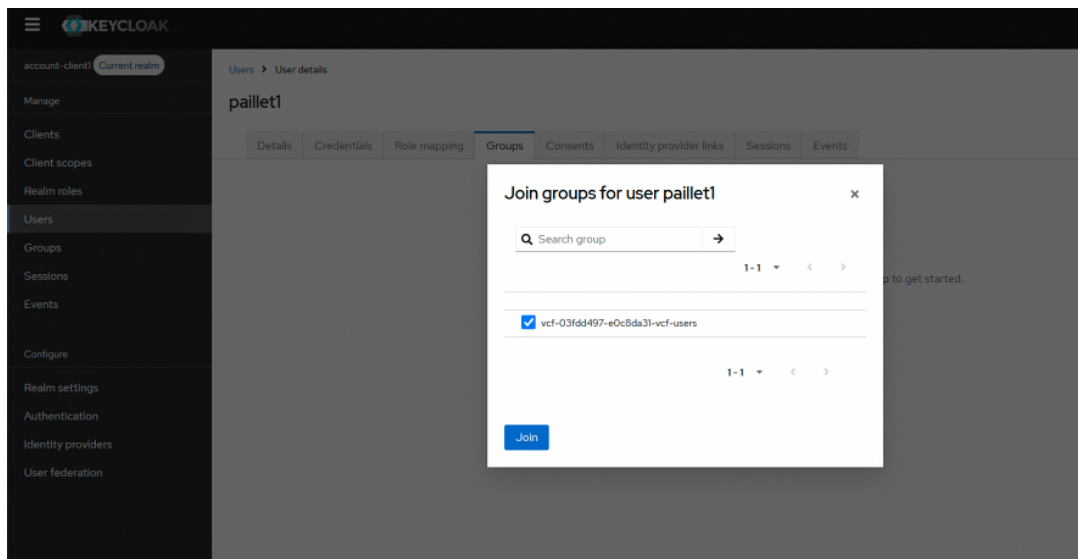
1. Log into the Keycloak admin console for the account realm (`account-{name}`)
2. Navigate to **Clients** and confirm the app's client-id is present
3. Navigate to **Clients** → select the app's client-id → **Roles** tab, and confirm the service-specific client roles exist (e.g., `vcf-users` — as defined in the service's `cloudstore.yaml`)
4. Navigate to **Groups** and confirm the corresponding groups were created (e.g., `vcf-<deployment-id>-vcf-users`)

2. Assign users to groups

Add end users to the appropriate groups to grant them access to the app:

1. Navigate to **Users** in the Keycloak admin console

2. Select the user
3. Go to the **Groups** tab
4. Click **Join Group** and select the relevant group



The user will now receive the associated client roles in their OIDC token when accessing the app. It is the app's responsibility to enforce authorization based on these roles.

! Do not assign roles directly to users

Always assign users to groups rather than assigning roles directly. CloudStore plans to integrate group-level authorization into the platform, and direct role assignments will not be supported by this future feature.

3. Verify end user access

1. Log into the [Landing Zone](#) as the end user
2. Select the account
3. Confirm the app appears in the list of authorized apps
4. Access the app and verify the expected permissions are active

If the app does not appear, check that:

- The user is a member of a group linked to the app's roles
- The app deployment completed successfully
- The Keycloak realm name matches the expected `account-{name}` pattern

4. Remove access

To revoke a user's access to an app:

1. Navigate to **Users** → select the user → **Groups** tab
2. Remove the user from the relevant group

The user will no longer see the app in the Landing Zone.

3.2.3 Further reading

- [Service authorization \(explanation\)](#)
- [Tenancy — Roles and groups](#)
- [Personas](#)
- CloudStore Service Packager documentation — authoring the `permission` block in `cloudstore.yaml`

4 Reference

4.1 Reference

Factual, look-it-up material: definitions, matrices, and operational characteristics of the CloudStore. Use these pages to check a precise detail rather than to learn a concept — for the conceptual picture, see [Explanation](#).

4.1.1 Concepts & definitions

Architecture Layers

The three-layer OPCP Core / CloudStore / Service stack and who owns each layer.

Personas

Definitions of each actor: DC operator, IT admin, service admin, end user.

Services

Definitions of a service, controller, and app, and how they relate.

Tenancy

The multi-tenant isolation model and account boundaries.

4.1.2 Capabilities & operations

Feature Reference

Feature matrix with status, required role, and prerequisites for each capability.

Platform runtime & deployment

How a platform's runtime reconciles workloads and how a controller / app is deployed.

Availability & Disaster Recovery

Availability characteristics, backup and DR mechanisms, and topology constraints.

Roadmap & Maturity

What CloudStore ships today versus what is still ahead.

4.1.3 Glossary

Glossary

Terms, abbreviations, and component names used across the documentation.

4.2 Concepts & definitions

4.2.1 Architecture Layers

The `Cloudstore` / `OPCP core` stack is organized in 3 distinct layers:

layer	Persona	internal name	description
OPCP Core	DC operator	layer 1	The hardware, tools and automations needed to operate your infrastructure with the help of globally reknown softwares (Openstack, Netbox)
CloudStore	Cloud Provider (IT Admin & service admins)	layer 2	The shelf-based software and management tools that allow you to provison and maintain cloud services
Landing Zone	End user / Accounts	layer 3	The deployed cloud services deployed for the need of your internal teams or clients

The CloudStore is an OVHcloud product dedicated to fast and easy install of Cloud Native solutions through a marketplace. The Landing Zone is commonly included in the CloudStore.

4.2.2 Persona Definitions

OPCP Core

DC operator

The DC operator manages the physical resources of the OPCP Core platform. They enroll hardware and ensure infrastructure availability.

Super Admin

The Super Admin configures OpenStack projects, assigns hosts and roles to projects, and manages IT admin access through OPCP Core's Keycloak.

Warning

The Super Admin is not currently implemented and is considered to be the same user as the DC operator.

CloudStore

IT admin

The IT admin has admin rights on the [CloudStore](#). They can create [accounts](#), activate [services](#), and deploy [apps](#).

Service admin

The service admin manages a specific [service](#) on the [CloudStore](#). They can activate their service and deploy apps for it.

Warning

The IT admin and service admin roles are not currently differentiated — every authenticated CloudStore user has rights on all services. However, OpenStack project scoping is enforced.

Landing Zone

Account admin

The account admin manages the configuration of their [account](#). They are the first user created with the `account-admin` role when the account is provisioned.

Account

An account represents a company or a department of a company. It is created by an [IT admin](#) in the CloudStore. It associates a set of [end users](#) to a set of [apps](#) that will be available for them through the Landing Zone.

Each [service](#) is attached to an account before deployment.

End user

The end user accesses [apps](#) through the Landing Zone. End users authenticate via their account's Keycloak realm and can only access apps deployed for their account.

End users cannot be shared between [accounts](#).

Permission matrix

The following matrix covers permissions on OpenStack projects, accounts, services, and apps (not on L1/L2 user management).

Persona	Enroll hardware	Configure OpenStack projects	Create account	Activate service	Deploy an app	Configure account	Access i
DC operator	X						
Super Admin		X					
IT admin			X	X	X		
Service admin			X	X*	X*		
Account admin						X**	
End user							X***

- * only if admin of this service
- ** only if admin of this account
- *** only if in the account for which the app is deployed

4.2.3 Service Definitions

App

An app is a set of rights on one or more resources of the same type. In the CloudStore context, an app is the workload part of a [service](#).

- There can be none, one or more apps per [controller](#).
- Apps cannot be shared between [accounts](#).
- Through the CloudStore UI, an app is accessible by the [IT admin](#) and the [service admin](#).

See [Features Reference — Deploy an app](#) for the detailed deployment specification.

Synonymous: [dataplane](#)

Cloud App

Synonymous: [Service](#)

Controller

A controller is the manager part of a [service](#). From the controller, [apps](#) can be started.

Deploying a controller activates its [service](#), making it available through the CloudStore UI for the [IT admin](#) and the [service admin](#). A controller can be deployed by the IT admin alone. Once the service is activated, [apps](#) can be deployed for it.

See [Features Reference — Activate a service](#) for the detailed deployment specification.

Synonymous: [controlplane](#)

Controlplane

Synonymous: [Controller](#)

Dataplane

Synonymous: [App](#)

Instance

An instance is a reconciliation loop of a [service controller](#) and [apps](#).

Service

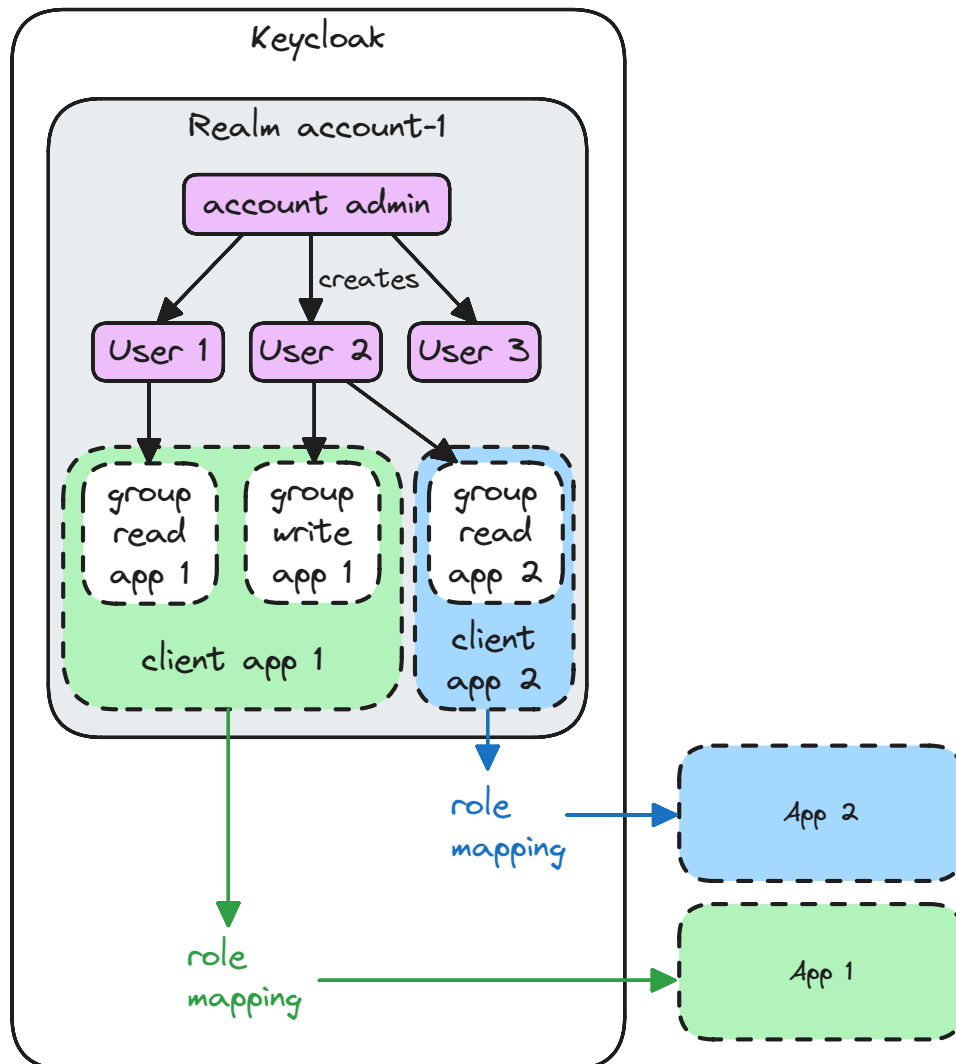
A service is a solution integrated and made available through the [CloudStore](#). A service is multiapp by design and includes one or more [controllers](#) and one or more [apps](#).

Synonymous: [Cloud App](#)

4.2.4 Tenancy

Multi-tenant isolation

CloudStore handles multitenancy natively. When an app is deployed, an account is chosen and the app is only available to the allowed users of that account.



Roles and groups

When an app is deployed, `roles` and `groups` are created in the account's Keycloak realm. These roles and groups are defined by the service — for example `read`, `write` and `admin` groups that grant the corresponding app permissions to users.

Companies can create functional groups that reflect their organizational structure, with child groups mapping to the technical groups above.

Do not assign roles directly to users

Although it is possible to assign roles directly to users in Keycloak, we strongly advise against this, as we plan to integrate group-level authorisation directly into the Cloudstore in the future.

Account

An [account](#) represents a company or a department of a company. Each account corresponds to a dedicated Keycloak realm (named `account-{name}`), providing complete IAM isolation between tenants.

All Keycloak realms matching the prefix `account-` are counted as accounts. For each account, the admin is determined by the first user with the `account-admin` role in the realm.

Scope of tenant isolation

Isolation today is enforced at the **IAM realm** and **Kubernetes namespace** levels only. There is **no per-tenant network policy, storage segregation, or resource quota** — tenant workloads share the node's compute, storage I/O and network paths, so a noisy-neighbor effect is possible. Per-account quotas are a planned feature.

See [Features Reference — Create an account](#) for the detailed account creation specification.

4.3 Capabilities & operations

4.3.1 Features Reference

Feature matrix

Feature	Status	Role required	Prerequisites	Description
Access the CloudStore	Available	IT admin, Service admin	Keycloak L2 account	Authenticate and access the CloudStore management interface
Activate a service (deploy controller)	Available	IT admin, Service admin*	CloudStore access, available hosts	Deploy the controller (management domain) of a service on designated hosts
Create an account	Available	IT admin	CloudStore access	Create a tenant account with its own Keycloak L3 realm and admin user
Deploy an app	Available	IT admin, Service admin*	Active controller, existing account	Deploy a service app scoped to a specific account
Add / remove hosts	Available	IT admin, Service admin*	Active controller or app	Scale capacity by adding or removing hosts from an instance
Backup / restore services	Planned	—	—	Protect and recover service state
Upgrade services	Planned	—	—	Upgrade service versions
Apply quotas	Planned	—	—	Set resource consumption limits per account
Track user consumption	Planned	—	—	Monitor resource usage across accounts
Unified observability	Planned	—	—	Monitoring and metrics for end users
Secret management (KMS)	Planned	—	—	Centralized key and secret management
Track own consumption	Planned	—	—	End user consumption visibility

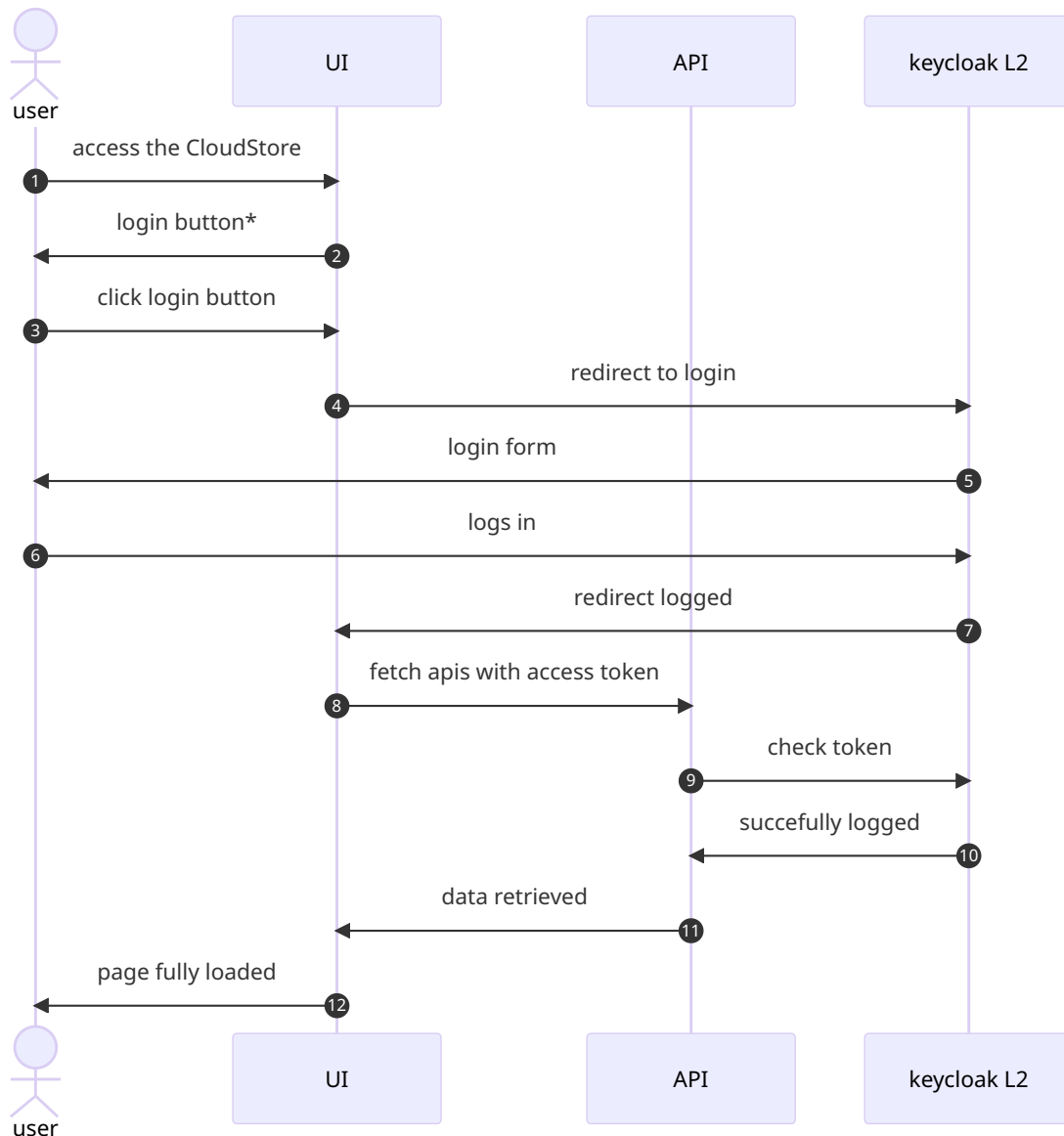
* only if admin of this service

Feature specifications

Access the CloudStore

Components: UI, API, Keycloak L2

Authentication flow:



* the Keycloak URL is provided through an environment variable.

Behavior: Synchronous. The UI redirects to Keycloak L2 for OIDC authentication, then uses the obtained access token for all subsequent API calls.

Activate a service (deploy controller) / Deploy an app

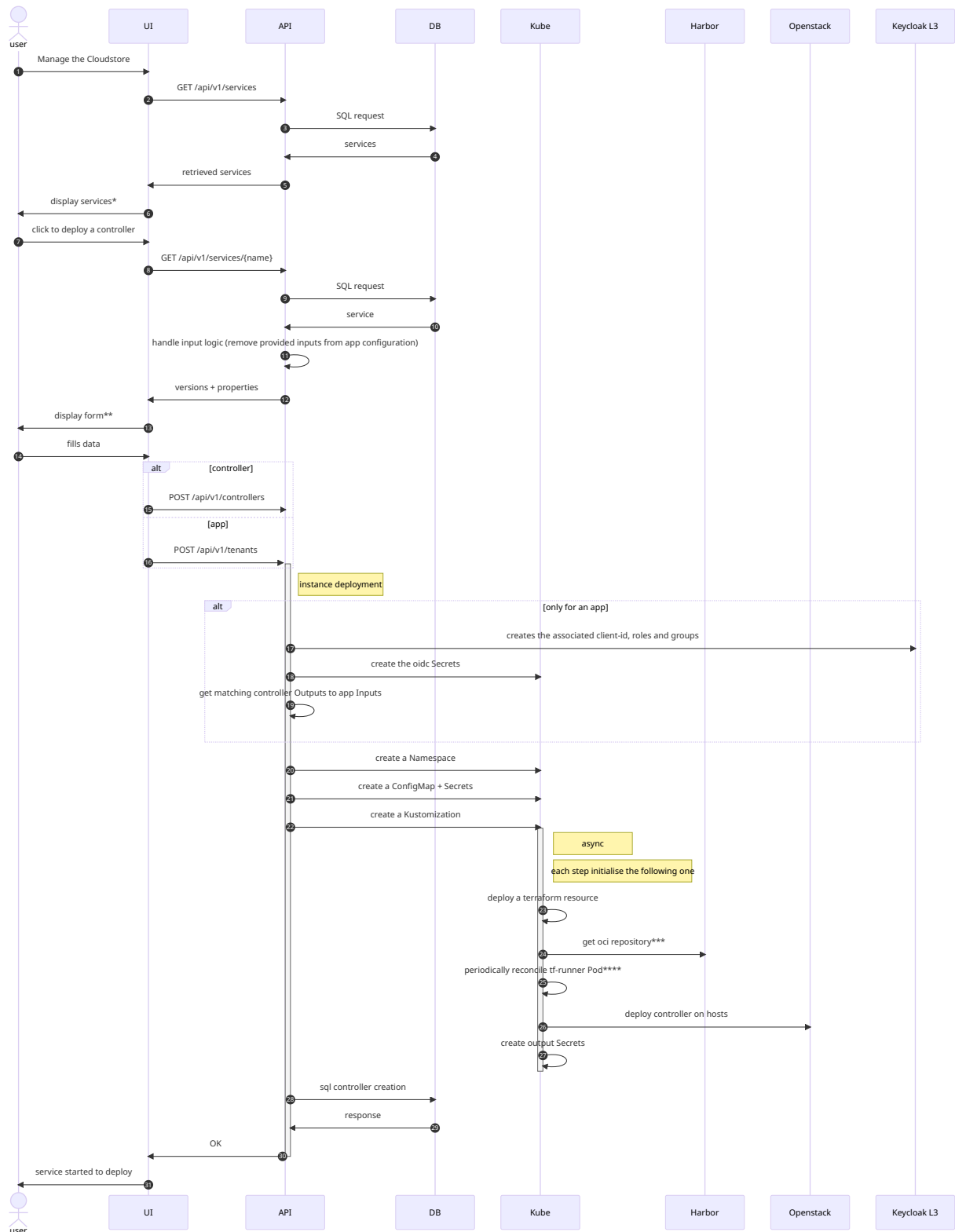
Components: UI, API, DB, Kubernetes, Harbor, OpenStack, Keycloak L3

Endpoints:

- Service listing: `GET /api/v1/services`
- Service detail: `GET /api/v1/services/{name}`

- Controller deployment: `POST /api/v1/controllers`
- App deployment: `POST /api/v1/tenants`

Deployment flow:



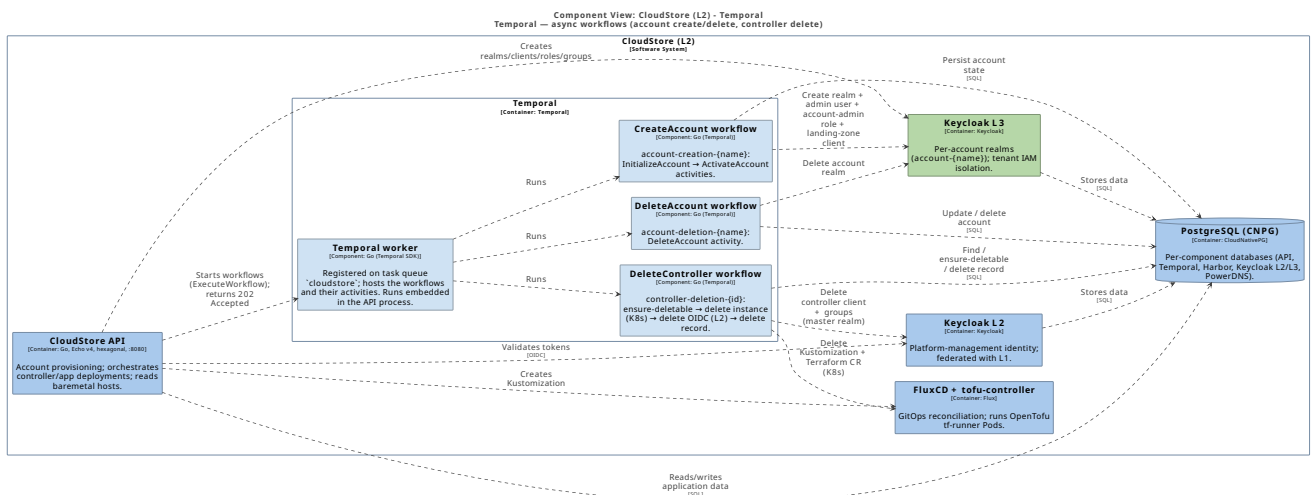
- * display a "Activate the service" button for services not yet activated
- ** name, version and properties from the `cloudstore.yaml` manifest
- *** use the OCI repository containing the Terraform resources
- **** get its configuration from previously created ConfigMap and Secrets

Behavior: The API call returns immediately (`OK`), but the actual deployment is **asynchronous**. Kubernetes orchestrates the deployment through a Kustomization that triggers a Terraform runner, which provisions resources on OpenStack.

App-specific behavior: When deploying an app, the API additionally creates Keycloak L3 resources (client-id, roles, groups) in the target account's realm and injects OIDC secrets into Kubernetes before starting the deployment.

Create an account

Components: UI, API, Keycloak L2, Keycloak L3

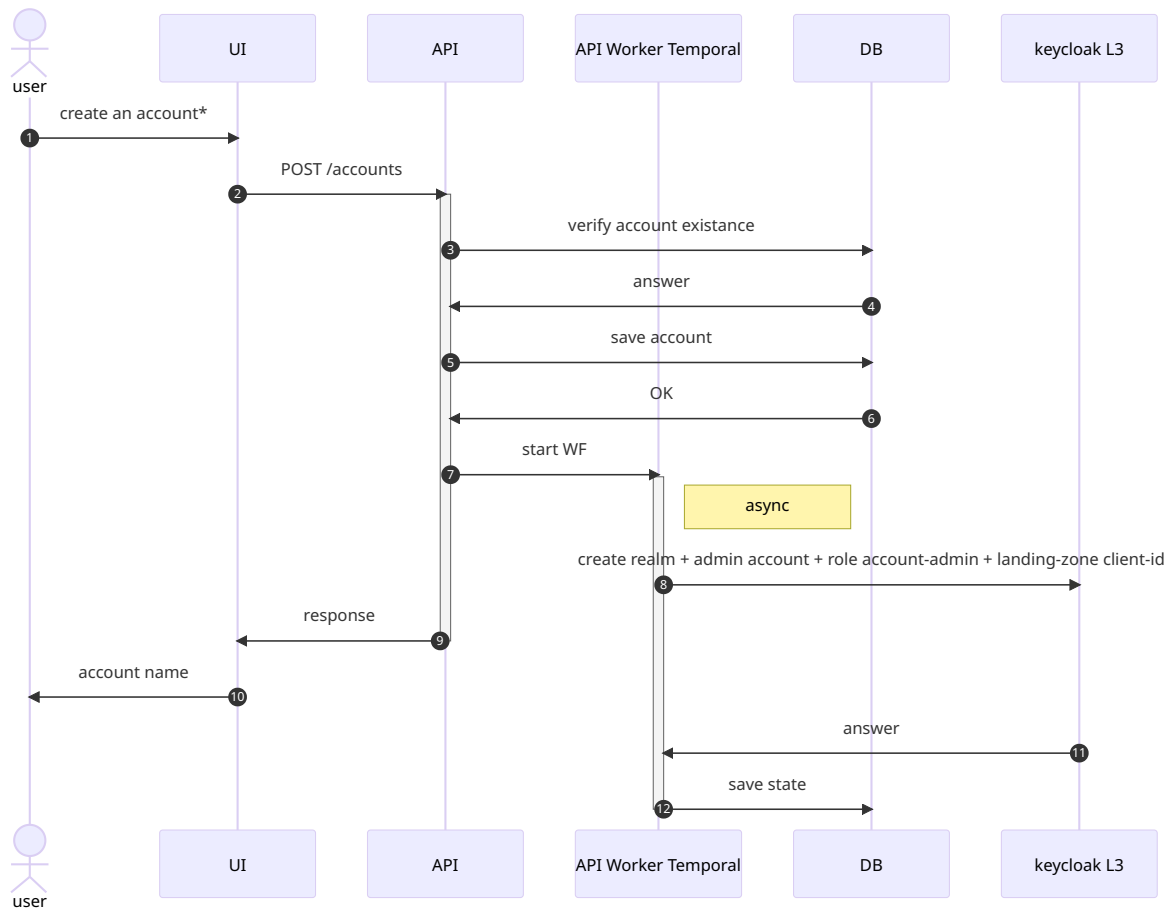


Account creation runs on Temporal — component view from the [architecture overview](#).

Endpoint: `POST /api/v1/accounts`

Required input: account name, admin full name, admin email.

Account creation flow:



* provide account name, admin full name and email.

Behavior: Asynchronous. The API records the account and returns immediately; the Keycloak provisioning below runs on a Temporal worker. It performs the following operations:

1. Creates a Keycloak realm named `account-{name}` (the name serves as account identifier)
2. Creates a realm role named `account-admin` containing the composite `realm-admin` role
3. Creates a user with:
 - Username: `{admin.email}`
 - Email: `{admin.email}`
 - FirstName: `{admin.firstName}`
 - LastName: `{admin.lastName}`
 - Temporary password configured via `ACCOUNT_ADMIN_FIRST_PASSWORD` environment variable
4. Assigns the `account-admin` role to the user
5. Creates a `landing-zone` client-id in the realm

Account listing: All Keycloak realms matching the prefix `account-` are counted as accounts. The admin is determined by the first user with the `account-admin` role. The admin may be null if the user was deleted (logged as warning).

Add / remove hosts

Components: UI, API, DB, Kubernetes, OpenStack

Endpoints:

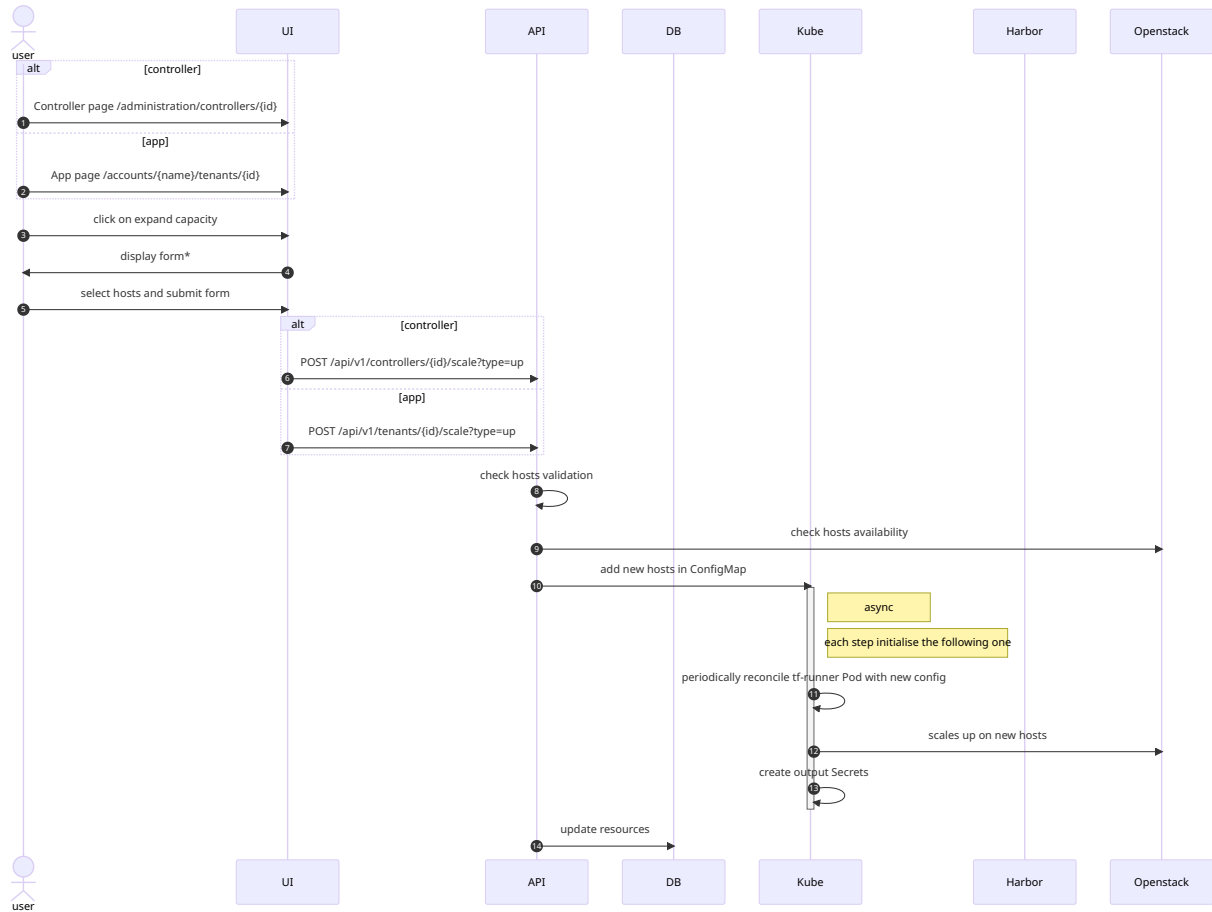
- Controller: `POST /api/v1/controllers/{id}/scale?type=up`

- App: `POST /api/v1/tenants/{id}/scale?type=up`

UI paths:

- Controller: `/administration/controllers/{id}`
- App: `/accounts/{name}/tenants/{id}`

Scaling flow:



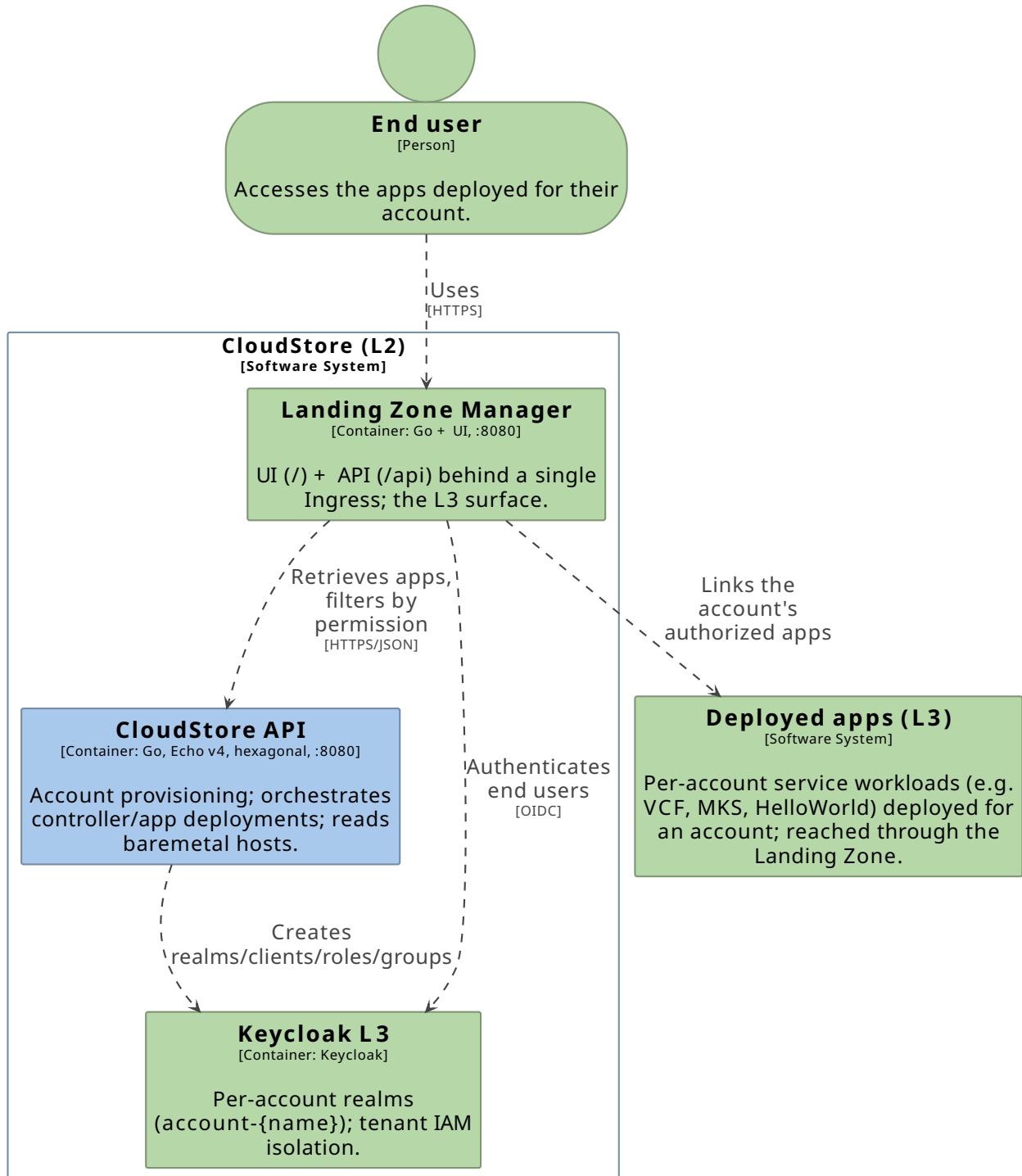
* display a table form with the ability to select available hosts to expand the capacity on.

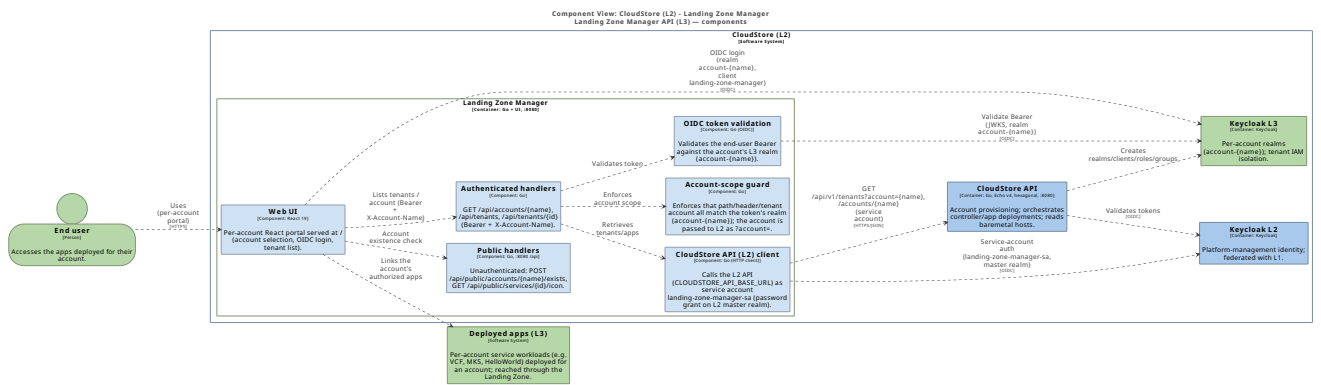
Behavior: The API validates hosts and checks their availability on OpenStack, then updates the ConfigMap in Kubernetes. The actual scaling is **asynchronous** — Kubernetes reconciles the Terraform runner with the new configuration to provision resources on the new hosts.

Access the Landing Zone

Components: Landing Zone UI, API L3 (LZM), API L2 (CloudStore), Keycloak L3

Container View: CloudStore (L2) Landing Zone Manager (L3) surface



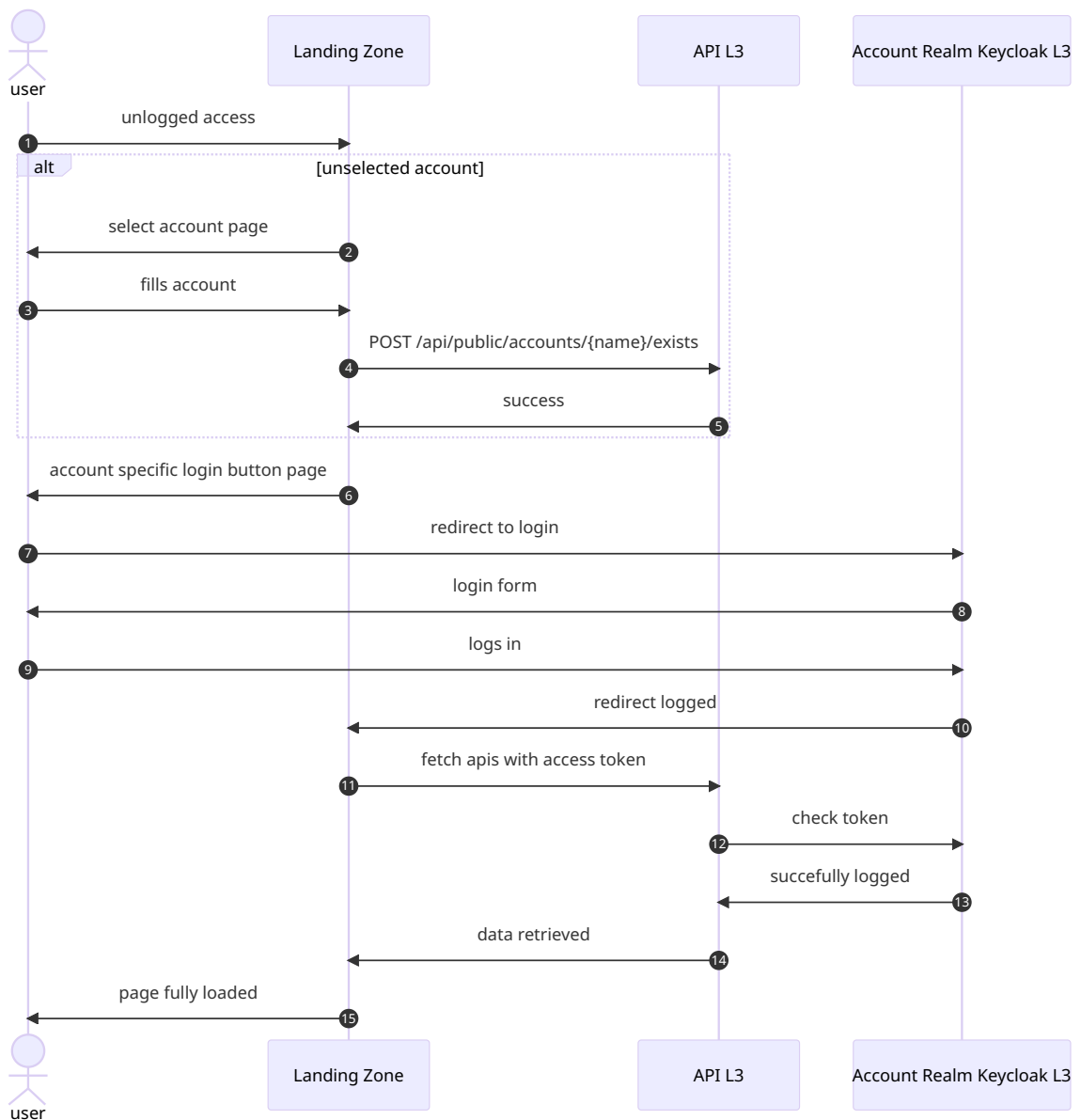


Landing Zone Manager container & API-component views — part of the [architecture overview](#).

Endpoints (LZM external API — note there is no `/v1`; that prefix belongs to the upstream CloudStore L2 API the LZM proxies to):

- Account existence check: `POST /api/public/accounts/{name}/exists`
- Tenant (app) listing: `GET /api/tenants` (requires `Authorization: Bearer` and an `X-Account-Name` header)
- Tenant detail: `GET /api/tenants/{id}`

Login flow:



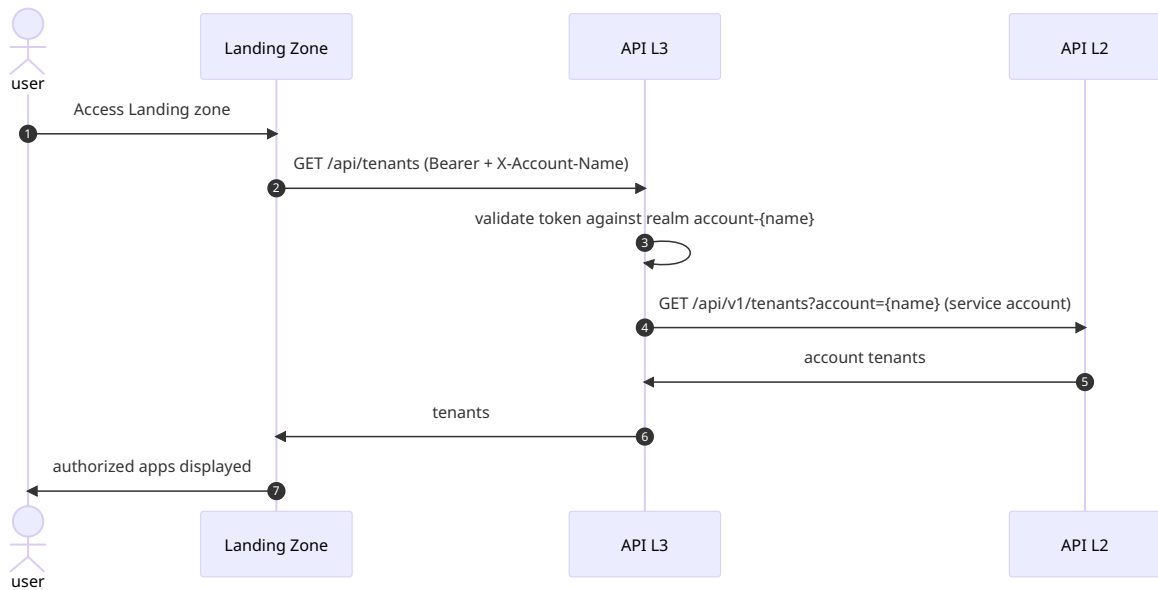
Quick login — the `account-name` query parameter

The account can be pre-selected via an `account-name` query parameter on the Landing Zone URL, which skips the *select account page* step of the login flow and takes the user straight to the account-specific login button page:

```
https://landing-zone-manager-0.<domain name>/?account-name=<account>
```

This parameter is **added automatically when the Landing Zone is opened from the CloudStore** (which already knows the account context), so end users don't type it by hand. It can also be used manually to deep-link to a specific account's login.

App access flow:



Behavior: The Landing Zone first requires account selection, then authenticates the user via Keycloak L3 (the account-specific realm `account-{name}`). API L3 validates the user's token against that realm — binding the caller to the account — then retrieves **that account's tenants** from API L2 (CloudStore), scoped by `?account={name}` and fetched with a dedicated service account, and returns them as the account's authorized apps.

4.3.2 Platform runtime & deployment

This page is a factual reference for how a CloudStore platform's runtime reconciles its workloads and how a controller / app is deployed. For the conceptual picture, see [CloudStore architecture](#) and [Hosting the CloudStore](#).

Runtime & reconciliation

A CloudStore platform runs on a lightweight Kubernetes runtime and reconciles all of its workloads through GitOps.

- **Runtime** — a K3s (lightweight Kubernetes) cluster. The container network interface (CNI) is **Cilium**.
- **Cluster topology** — the cluster is **single-node**. There is no control-plane / etcd high availability and no node redundancy; each platform is a separate single-node cluster.
- **GitOps** — **FluxCD** reconciles Kustomizations. On customer / production platforms the GitOps content is shipped as **OCI artifacts**: Flux reads it from a local path on the node, so there is **no remote git host** involved on the platform itself.
- **IaC execution** — OpenTofu plans are reconciled by the **tofu-controller** (a Flux controller). Terraform state is stored as a **gzip-encoded Kubernetes Secret** in a per-deployment namespace named `deploy-<service>--<hash>` (with a further `deploy-<service>--<controller-hash>--<app-hash>` namespace per app).
- **Registry** — a platform-local **Harbor** OCI registry is a core service; services pull their images from it at deploy time.

 Where images come from on a deployed platform

Core-service images are added at installation time. Cloud-app images are imported later via the metapackage offline import, which keeps the platform self-contained.

Version pinning

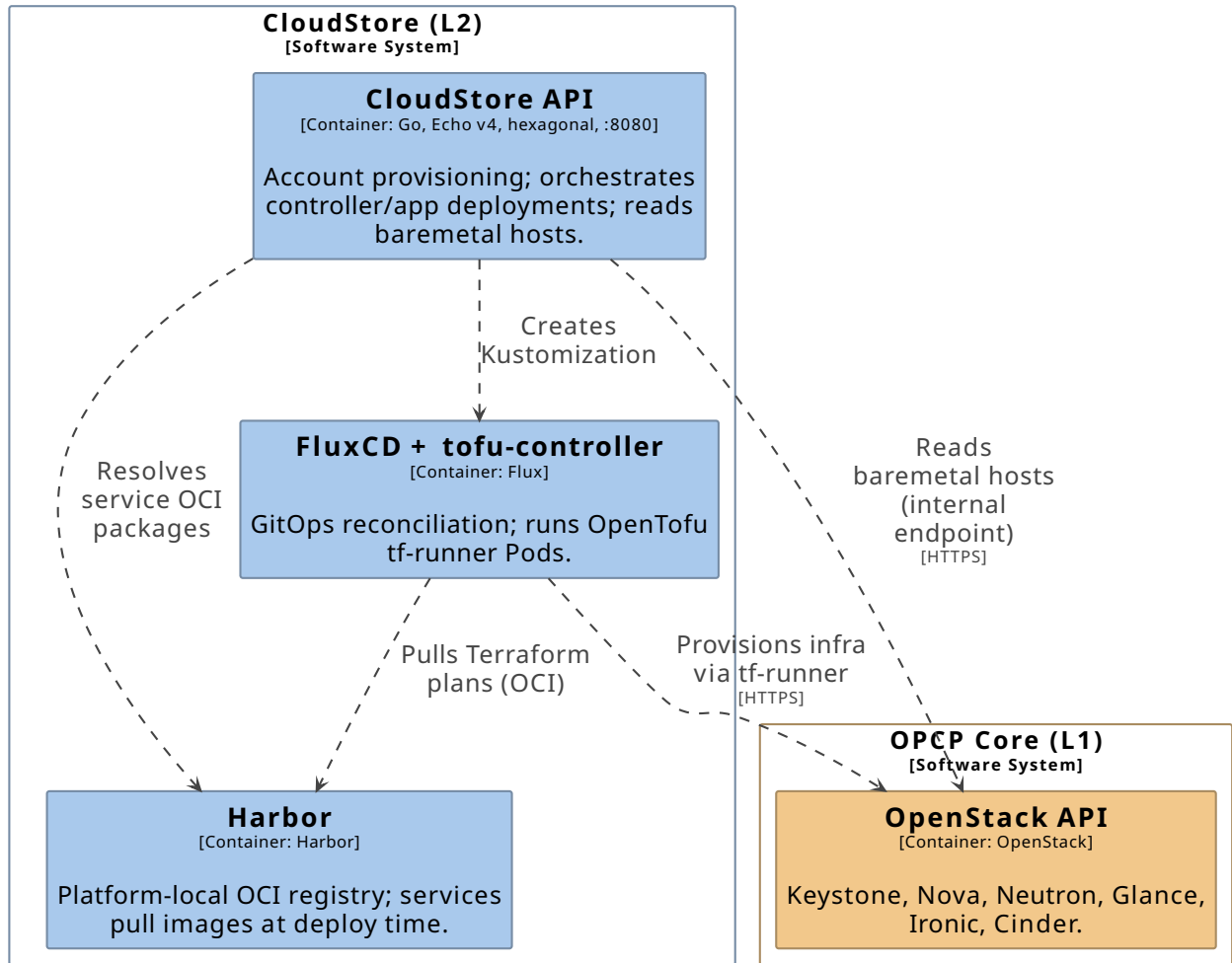
Component versions are pinned per **tagged release tarball**, which makes air-gapped builds reproducible. The tagged release is the source of truth for the exact versions, split by ownership into two tarballs:

Tarball	Pinned components
OPCP tarball	Shared / common platform components: K3s, Cilium, Traefik, Flux
<code>cloudstore-core</code> tarball	CloudStore-specific components: Keycloak, Harbor, tofu-controller, CNPG/Barman, and the API/UI

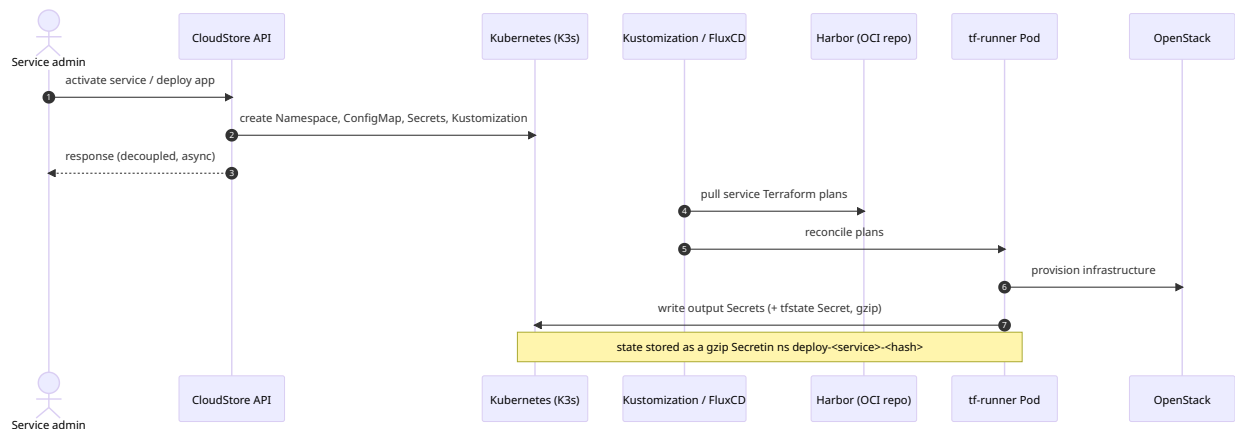
Deployment flow of a controller / app

When a service is activated or an app is deployed, the API creates the Kubernetes resources, then a Kustomization asynchronously pulls the service's Terraform plans from the OCI registry and reconciles them through a **tf-runner** Pod, which provisions the infrastructure and writes output Secrets.

Container View: CloudStore (L2) GitOps & Deployment — Flux / tofu-controller provisioning



GitOps & deployment container view — part of the [architecture overview](#).



4.3.3 Availability & Disaster Recovery

This page describes CloudStore's availability characteristics, its backup mechanism, and the current topology constraints. It is factual reference material; see [Architecture](#) for the conceptual model and [Tenancy](#) for isolation boundaries.

Availability characteristics

Property	Behaviour
Deployments	Asynchronous and non-blocking — the API decouples its response from provisioning, so deployment requests do not block on the underlying work.
App scaling	Apps scale independently of their controller.
Cluster topology	Single-node K3s cluster — no HA (see below).

Async, non-blocking deployments

When a deployment is requested, the API returns immediately and provisioning proceeds asynchronously in the background. Callers observe progress through the resulting resources rather than a blocking call, which lets deployment work scale without holding requests open.

Independent app scaling

An [app](#) scales independently of its [controller](#). Scaling the workload does not require scaling the manager part of the service.

Topology & single point of failure

The CloudStore control plane runs on a **single-node K3s cluster**. There is no control-plane / etcd HA and no node redundancy. Each environment is its own separate single-node cluster.

 The control plane is a single point of failure

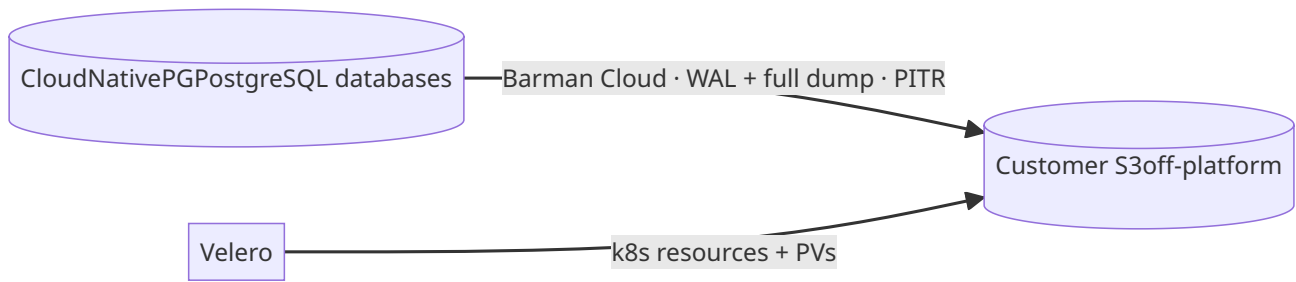
Because the cluster is single-node, the node hosting the CloudStore control plane is a **single point of failure**. The same node is also the **capacity ceiling** for what one CloudStore can manage — there is no horizontal scaling of the control plane.

Additional confirmed constraints:

- **One CloudStore per OPCP.** Running multiple CloudStores on a single OPCP is not supported today.
- **No documented capacity limits.** Capacity limits and known bottlenecks for a single CloudStore are not currently documented.

Backup mechanism

CloudStore ships with two complementary backup mechanisms.



Mechanism	Scope	Policy
CNPG + Barman Cloud	PostgreSQL application databases; point-in-time recovery (PITR) to S3	WAL archiving active, plus a daily full dump (schedule defined per service).
Velero	Kubernetes resources and persistent volumes	Daily backup, 30-day retention.

The backup target is customer-provided and off-platform

The S3 backup target is provided by the customer and lives off-platform — it is **not** hosted on the CloudStore node. It only needs to be reachable on the customer network, so backups work without public internet access (air-gap compatible).

Recovery & blast radius

Node-loss recovery

Recovery from loss of the CloudStore node is a **rebuild-and-restore** procedure:

1. Rebuild the node.
2. Restore Kubernetes resources and persistent volumes from **Velero**.
3. Restore PostgreSQL databases from **Barman Cloud** (point-in-time recovery from S3).

No formal RTO / RPO

No formal disaster-recovery targets (Recovery Time Objective / Recovery Point Objective) are defined. Only the CNPG/Barman and Velero mechanisms described above exist. Recovery duration and data-loss bounds are not contractually specified.

Blast radius of a control-plane outage

During a CloudStore control-plane outage, the impact is bounded:

- **Already-deployed tenant workloads keep running.** Tenant workloads (the [dataplane](#)) run on OPCP/OpenStack, not on the CloudStore node, and mostly continue operating during a control-plane outage.
- **New operations are blocked.** New deploy and manage operations against CloudStore are unavailable while the control plane is down.
- **Core-service dependencies may still be affected.** Workloads that depend on CloudStore-hosted core services — Keycloak (IAM), DNS, and ingress — may be affected even though their own compute keeps running.

Related pages

- [Architecture](#) — conceptual architecture and the async API model.
- [Architecture Layers](#) — layered reference of the platform.
- [Tenancy](#) — isolation boundaries and the shared single-node footprint.

4.3.4 Roadmap & Maturity

This page records **what CloudStore ships today versus what is still ahead**, so that forward-looking capabilities described elsewhere in the documentation are not read as current fact. Where a topic is described "as designed" in the [architecture](#) and reference pages, this page tells you how mature that capability actually is.

Items are grouped into four maturity buckets:

- **Already there** — implemented and running on real platforms.
- **Working on it** — under active design or development now.
- **Planned** — agreed direction, not started or not yet delivered.
- **Was discussed at some point** — proposed, parked, or rejected; no committed delivery.

Already there

- **CloudStore platform on K3s + GitOps** — a single-node K3s cluster, CNI provided by Cilium, continuously reconciled by FluxCD.
- **Everything-as-OpenTofu** — infrastructure and services are declared as OpenTofu and applied by tofu-controller, following a controlplane / dataplane deployment model.
- **Three-tier Keycloak IAM** — a layered identity model with one realm per account and federation between the platform and the upstream identity provider. See [Tenancy](#).
- **Core platform services** — OPA Gatekeeper, cert-manager, PowerDNS, ExternalDNS, CloudNativePG (CNPG), Harbor, Velero, Keycloak, Prometheus, Grafana, Loki and Alloy.
- **Networking** — Traefik ingress, PowerDNS with ExternalDNS for DNS, and cert-manager for TLS.
- **Data & backup mechanisms** — CNPG (CloudNativePG) with Barman Cloud point-in-time recovery to S3, and Velero for Kubernetes resources and persistent volumes. This is **platform-level only**. See [Data & Storage](#).
- **Service packaging** — the packaging toolchain ([cspkg](#) , Copier templates, package generation pipeline) with SemVer / Commitizen versioning.
- **Service install / uninstall** from the catalog.
- **Multi-tenant isolation** — one IAM realm and one Kubernetes namespace per tenant. See [Tenancy](#).
- **Encryption at rest** — provided by self-encrypting drives (SED).
- **Air-gapped PKI** — the issuing certificate authority is provided on-premise; there is no reliance on public ACME.
- **On-Premise hosting mode** — CloudStore running in the customer's own datacenter, fully air-gapped, including air-gapped version updates.

Working on it

- **Metapackage / dependency management** — a self-contained [tar.gz](#) that bundles a service package together with all of its external dependencies (OCI images, Docker images, VM images, Debian packages, S3 objects) for offline import. This is **under active design and not yet available**; today only OCI images and Docker images imported into Harbor are supported.
- **Unified observability and secret management (KMS)** for cloud users.

Planned

- **PaaS SecNumCloud (SNC) hosting mode** — CloudStore running on a SecNumCloud-qualified **Bare Metal Pod (BMPod)** in OVHcloud datacenters, with a per-customer dedicated control plane and a Landing Zone Manager for SNC users. This mode is **not yet delivered** — see the note below.

- **Service upgrade & rollback** — in-place upgrade and rollback of a deployed service; today the only path is to redeploy.
- **Backup / restore of tenant (per-app) state** — only platform-level point-in-time recovery and Velero backups exist today.
- **Quotas and consumption tracking** for cloud providers.
- **Audit logging** — centralized, who-did-what tracing across the API, identity provider and GitOps reconciliation (a SecNumCloud requirement); not implemented yet.
- **Internal service-to-service TLS** — in-transit encryption between internal services is not enforced today.
- **Observability replacements** — migrating Prometheus toward Mimir and Alloy toward its successor.
- **Alerting routing / on-call** — AlertManager routing and on-call paths are not yet defined.
- **Reusable management shell** — reusing the Manager micro-frontend architecture for a shared shell; currently a technical proposal, not implemented.

Was discussed at some point

- **DR targets (RTO / RPO)** — no formal recovery-time or recovery-point targets are defined today; recovery is via Velero and Barman restore only.
- **Multiple CloudStores per platform** — not supported; there is one CloudStore per platform today.
- **Threat model / trust-boundary document and a CloudStore-specific SecNumCloud control mapping** — neither has been produced yet.

About PaaS SecNumCloud (SNC / BMPod) references elsewhere

The PaaS SecNumCloud (SNC / BMPod) **hosting mode is planned and not yet delivered**. Wherever it appears in the rest of this documentation — deployment contexts, install triggers, SecNumCloud topics and the glossary — those statements describe the **intended design**, not a shipped capability. Read them against this roadmap entry until the mode is delivered.

4.4 Glossary

This glossary defines the terms, abbreviations, and component names used across the CloudStore documentation. Terms are grouped by topic. The **Referenced in** column links to the main pages and sections where each term is used. For the architecture as a set of diagrams, see [Architecture diagrams \(C4 model\)](#).

4.4.1 Platform, layers & hosting

Term	Definition	Referenced in
CloudStore	The CloudStore is an OVHcloud product dedicated to fast and easy deployment of Cloud Native solutions through a marketplace. It constitutes Layer 2 of the OPCP stack. See Architecture.	Architecture Layers CloudStore architecture Architecture diagrams (C4) Hosting
OPCP	OnPrem Cloud Platform. The full OVHcloud on-premises product. Its foundation layer is OPCP Core (Layer 1).	Hosting Architecture Layers
OPCP Core	The infrastructure layer (Layer 1) providing hardware and tools for operating the infrastructure using OpenStack and Netbox. See Architecture.	Architecture Layers Networking — Connectivity with OPCP Core Hosting
Landing Zone	The deployed cloud services interface accessible to end users and accounts. It constitutes Layer 3 of the OPCP stack, commonly included in the CloudStore. Also called the Landing Zone Manager (LZM). See Architecture.	Architecture Layers Authentication — Keycloak L3 Features — Access the Landing Zone
L1 / L2 / L3	Shorthand for the three layers of the platform: Layer 1 (OPCP Core), Layer 2 (CloudStore), and Layer 3 (Landing Zone).	Architecture diagrams (C4) Architecture Layers Hosting — Where CloudStore fits
BMPod	Bare Metal Pod. A SecNumCloud-qualified hosting undercloud used for the SecNumCloud (SNC) PaaS mode.	Hosting Roadmap
SNC	SecNumCloud. The French ANSSI security qualification, and the corresponding secure hosting mode.	Hosting — SNC CloudPlatform Security — SecNumCloud Roadmap

Term	Definition	Referenced in
OPCP On-Premise	A hosting context in which the platform runs inside the customer's own datacenter with no Internet connectivity (air-gapped), receiving updates through offline means.	Hosting — On-Premise Cloud Platform
Air-gap	An operating mode with no outbound Internet access. Every artifact needed at deploy time is provisioned inside the platform ahead of time.	Security — Air-gap operation , Networking — Air-gapped operation , Packaging & delivery — Metapackage

4.4.2 Personas / roles

Term	Definition	Referenced in
DC operator	Data-center operator. A Layer 1 persona who enrolls hardware into OPCP Core.	Personas — DC operator , Features — How features build on each other
Super Admin	A Layer 1 persona who configures OpenStack projects, hosts, and roles.	Personas — Super Admin
IT admin	A Layer 2 persona who authenticates through CloudStore (Keycloak) and creates and manages accounts.	Personas — IT admin , Features — Why two distinct roles
Service admin	A Layer 2 persona who activates services (controllers) and deploys apps for accounts.	Personas — Service admin , Authorization — Two levels
End user	A Layer 3 persona who consumes the apps they are authorized to use through the Landing Zone.	Personas — End user , Authentication — Keycloak L3
Cloud provider	The customer who operates CloudStore and offers services to their own customers. Sometimes simply called the provider.	Roles overview — Cloud providers , Features — Why two distinct roles

4.4.3 Service, tenancy & packaging

Term	Definition	Referenced in
Service	A solution integrated and made available through the CloudStore. A service is multiapp by design and includes one or more controllers and one or more apps. See Service .	Services — Service , Service package
Controller	The manager part of a service (also called controlplane). From the controller, apps can be started. A controller is a cross-tenant orchestrator, deployed once and shared across accounts. See Controller .	Services — Controller , Features — The controller/app pattern
App	The workload part of a service (also called dataplane). A set of rights on one or more resources of the same type. Apps are scoped to a single account and cannot be shared between accounts. See App .	Services — App , Features — The controller/app pattern
Instance	A reconciliation loop of a service controller and apps. See Instance .	Services — Instance
Enabler service	A third service category (ADR-0017, alongside core and customer services) exposing a mutualised capability shared across all accounts of an install. An enabler has a shared controlplane deployed once at install on a dedicated L3 nodepool of the controller cluster, plus per-account dataplanes ; customer services opt in by declaring a dependency in <code>cloudstore.yaml</code> (the <code>enablers:</code> block, semver-pinned). Packaged under the <code>cloudstore-service-enabler-*</code> repo prefix	See also CloudStore architecture — Core services
Account	A company or department representation in the CloudStore. Associates a set of end users to a set of apps. Each account is an isolated tenant and corresponds to a dedicated Keycloak realm. See Account .	Personas — Account , Tenancy , Features — Create an account
Multi-tenancy	Native isolation between accounts within the same CloudStore instance. Each account gets its own Keycloak realm, and apps are scoped to a single account. See Tenancy .	Tenancy , Security — Identity and access isolation
Service package	An OCI artifact that packages a deployable managed service, including its controlplane and dataplane infrastructure code, Helm charts, and a descriptor.	Service package

Term	Definition	Referenced in
Realm	A Keycloak isolation boundary. CloudStore creates one realm per account at Layer 3.	Tenancy — Account , Authentication — Keycloak L3
cloudstore.yaml	The service manifest (descriptor) that transmits a service's specification to CloudStore. It uses a Kubernetes-style schema (for example, <code>version: v1alpha2</code>).	Service package — Structure , Authorization , Setup service authorization
Manifest version	The schema version of <code>cloudstore.yaml</code> (for example, <code>v1alpha2</code>). Distinct from the service's own semantic version.	Service package — Versioning & publishing
Metapackage	A <code>tar.gz</code> bundle containing a service package together with all of its external dependencies, for import into an air-gapped platform.	Packaging & delivery — Metapackage , Roadmap — Working on it
cspkg	The packaging command-line tool used to build and publish service packages.	Roadmap — Service packaging
tf-updater	A helper that localizes Terraform modules and Helm charts into an OCI service package.	Service package
Copier	A templating tool that scaffolds a new service repository and keeps it up to date with the template.	Service package , Roadmap
commitizen	A conventional-commits and versioning tool that drives semantic versioning (also referred to as <code>cz</code>).	Service package — Versioning & publishing , Roadmap
skopeo	A tool that mirrors the container images declared by a service into Harbor.	Service package
OCI_IMAGE_TAG	The version tag applied to a package, following semantic versioning (for example, <code>X.Y.Z</code> , or a pre-release or development suffix).	—
SemVer	Semantic Versioning 2.0, the versioning scheme used for service packages.	Service package — Versioning & publishing

Term	Definition	Referenced in
OCI	Open Container Initiative. The image and artifact format used for packages and images.	Service package , Platform runtime & deployment

4.4.4 Runtime, GitOps & IaC

Term	Definition	Referenced in
Kubernetes	The container orchestration platform used by CloudStore to manage service deployments through Kustomizations and Terraform runners.	CloudStore architecture — Technical concepts , Runtime & reconciliation
K3s	The lightweight, single-node Kubernetes distribution that CloudStore runs on.	CloudStore architecture — Technical concepts , Runtime & reconciliation , Availability — Topology
Cilium	The container networking layer (CNI) used by the cluster.	Networking — CNI , Runtime & reconciliation
FluxCD	The GitOps engine that reconciles Kustomizations from the Git repositories.	CloudStore architecture — Technical concepts , Runtime & reconciliation
Kustomization	A Flux resource that pulls and reconciles a service's Terraform plans.	Deployment flow , Data & Storage — Where state lives
tofu-controller	A Flux controller that reconciles OpenTofu plans. Also referred to as the Terraform controller.	CloudStore architecture — Technical concepts , Runtime & reconciliation
OpenTofu	The infrastructure-as-code engine used to run service plans (also written Terraform/OpenTofu).	CloudStore architecture — Technical concepts , Runtime & reconciliation
tf-runner	A pod that executes a Terraform plan and provisions resources on OpenStack.	Deployment flow , Networking — OpenStack API access
csinstall	The installer that bootstraps CloudStore into a bare K3s cluster.	CloudStore architecture
irobox	The Terraform modules that create the host and install CloudStore.	CloudStore architecture

Term	Definition	Referenced in
Temporal	A workflow engine used by the CloudStore API to run asynchronous work.	CloudStore architecture — Platform components , Data & Storage — Application data , Features — Create an account
tfstate	Terraform state, stored as a compressed Kubernetes Secret.	Data & Storage — Where state lives , Runtime & reconciliation

4.4.5 Core services

Term	Definition	Referenced in
Keycloak	Open source identity and access management solution. Used at two levels: L2 (CloudStore authentication) and L3 (per-account Landing Zone authentication). Also deployed at L1 (OPCP).	Authentication , Security — Identity and access isolation , Tenancy
IAM (Identity and Access Management)	The system managing identities and access permissions. In CloudStore, IAM is implemented through Keycloak with a federation model between L2 to L1 and L3 to L2.	Authentication , Authorization , Security — Identity and access isolation
Harbor	The platform-local, OCI-compatible container and artifact registry that runs as a CloudStore core service; services pull their images and Terraform/OCI artifacts from it at deploy time. Distinct from the central packaging registry, where packages and images are published.	Runtime & reconciliation
CNPG	CloudNativePG. The PostgreSQL operator that backs the databases of the core services.	Data & Storage — Application data , Availability — Backup & DR
Barman Cloud	The backup plugin used with CloudNativePG for S3 backups and point-in-time recovery.	Data & Storage — Backups , Availability — Backup & DR
Velero	A tool that backs up Kubernetes resources and persistent volumes to S3.	Data & Storage — Backups , Availability — Backup & DR
PowerDNS	The authoritative DNS server for the platform's domain.	Networking — DNS , Data & Storage
ExternalDNS	A component that watches Ingress resources and writes the corresponding DNS records into PowerDNS.	Networking — Record automation
cert-manager	A component that issues and automatically renews the TLS certificates on every Ingress.	Networking — TLS and PKI , Security — PKI and TLS trust

Term	Definition	Referenced in
trust-manager	A component that distributes certificate-authority bundles across the cluster.	Networking — TLS and PKI
OPA Gatekeeper	A policy-enforcement component that validates and mutates resources through admission webhooks.	Security — Policy enforcement , Networking — CA auto-injection
Traefik	The built-in ingress controller provided by K3s.	Networking — Ingress and external exposure , Observability — Metrics
Observability stack	The set of tools providing metrics, dashboards, logs, log collection, and alerting (Prometheus, Grafana, Loki, Alloy, and AlertManager).	Observability
Mimir	The announced replacement for Prometheus.	Observability — Metrics , Roadmap — Planned

4.4.6 OpenStack & infrastructure

Term	Definition	Referenced in
OpenStack	The infrastructure-as-a-service platform used by OPCP Core (Layer 1) for managing physical resources, projects, and hosts, and that CloudStore provisions on.	Networking — Connectivity with OPCP Core , CloudStore architecture , Hosting
Keystone	OpenStack's identity and authentication service.	Networking — Connectivity with OPCP Core
Nova	OpenStack's compute service.	Networking — OpenStack API access
Neutron	OpenStack's networking service.	Networking — OpenStack API access
Glance	OpenStack's image service, which stores VM images.	Networking — OpenStack API access , Packaging & delivery — Metapackage
Ironic	OpenStack's bare-metal provisioning service.	Networking — OpenStack API access
Cinder	OpenStack's block-storage service.	Networking — OpenStack API access
Barbican	OpenStack's secrets and key-management service.	—
Application credential	The OpenStack credential CloudStore uses to call the OpenStack API.	Networking — OpenStack API access
Ceph	The block-storage backend used by the platform.	—
SED	Self-encrypting drive. A mechanism providing encryption of data at rest.	Security — Encryption at rest , Roadmap

4.4.7 Networking, security & PKI

Term	Definition	Referenced in
TSIG	A transaction signature that secures dynamic DNS updates.	Networking — DNS
RFC 2136	The dynamic DNS update protocol that ExternalDNS uses to update PowerDNS records.	Networking — DNS
ACME / DNS-01	A certificate-issuance protocol and its DNS-based challenge, satisfied through the PowerDNS webhook. No public ACME service is used in air-gapped mode.	Networking — TLS and PKI , Security — PKI and TLS trust
ClusterIssuer	A cert-manager issuer representing a certificate authority (for example, the CloudStore CA, Customer CA, or OPCP CA).	Networking — TLS and PKI
CA	Certificate Authority. A trust anchor. In air-gapped mode the issuing CA is provided by OPCP Core (Layer 1).	Networking — TLS and PKI , Security — PKI and TLS trust
SOPS	A tool used to encrypt secrets stored in the infrastructure Git repositories, using age or GPG keys.	Security — Secret management
OOB	Out-of-band. The management (administration) network interface on the CloudStore host.	Networking — Host network model